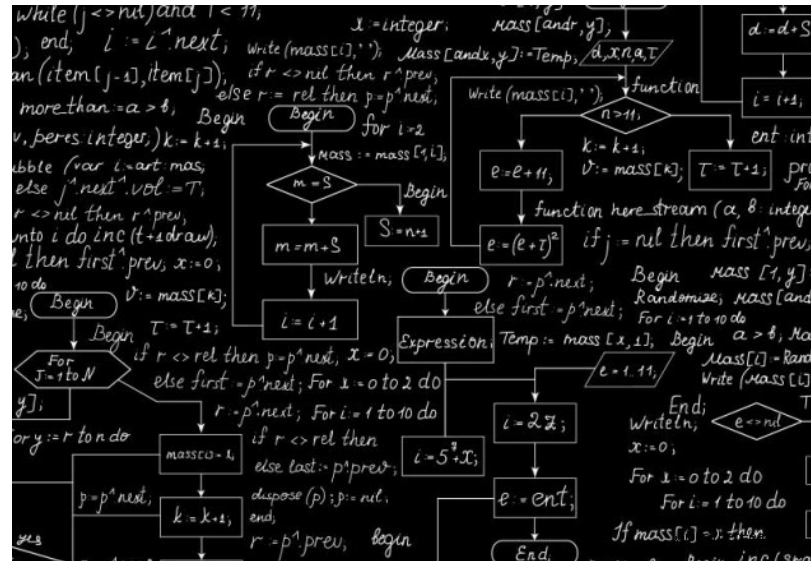


© 2010 Pearson Education, Inc. or its affiliate(s). All rights reserved. Pearson Education, Inc., publishing as Pearson Benjamin Cummings, 101 Philip Drive, Assinippi Park, New York, NY 10984-2135



1. Ritmada Jayida sayilar



El Harezmi - Abu Ja'far Mohammed Ibin Musa Al-Khowarizmi (MS 780-850)

Algoritma kavramı
fonksiyonlar
kavramı bulunur

- ❑ Algoritma kavramı
- ❑ Temel veri yapıları, temel problemler
- ❑ Fonksiyonların büyümesi, Asimptotik notasyonlar
- ❑ Algoritma karmaşıklığı ve analizi
 - İteratif ve özyinelemeli algoritmaların analizi

Algoritma

Tanım: Algoritma, hesaplama yapmak veya bir problemi çözmek için sonlu (finite) kesin (precise) komutlar/buyruklar/emirler (instructions) kümesidir.

Örnek: Sonlu bir tam sayı dizisinde maksimum değeri bulmak için bir algoritmayı yazınız.

Çözüm: Aşağıdaki işlemler sırasıyla uygulanır

1. Dizinin ilk elemanı geçici maksimum değer olarak seçilir
2. Bir sonraki eleman geçici maksimum ile karşılaştırılır
 - Eğer sayı geçici maksimumdan büyükse, geçici maksimumu bu sayıyı atanır
3. Dizin son elemanına kadar önceki adım tekrarlanır
4. Algoritma sonlandığında dizideki en büyük sayı bulunur

Algoritma nasıl belirtilir

- ❑ Algoritma adımları Türkçe, İngilizce veya sözde kodla (Pseudocode), akış şeması veya bir programlama dili ile açıklanabilir.
- ❑ Sözde kod, adımların İngilizce açıklaması ile bu adımların bir programlama dili kullanılarak kodlanması arasında ara bir basamaktır.
- ❑ Sözde kod, algoritmayı uygulamak için kullanılan gerçek programlama dilinden bağımsız olarak, algoritma kullanarak bir problemi çözmek için gereken zamanı analiz etmeye yardımcı olur.
- ❑ Programcılar, bir programı kodlarken sözde koddaki algoritmanın açıklamasını kullanabilir.
- ❑ Sözde kod, doğal bir dile doğrudan bağımlı olduğundan Akış Şeması algoritmanın görsel olarak ifade edilmesinde ortak bir dil elde etmeyi sağlar.
- ❑ Sözde kod veya Akış şemasına ihtiyaç olmayan durumlarda, Algoritma herhangi bir programlama diliyle de kodlanabilir.

Algoritmaların Özellikleri

- ❑ **Giriş (Input):** Bir algoritma, belirli bir kümeden giriş değerlerine sahiptir.
- ❑ **Çıkış (Output):** Algoritma, girdi değerlerinden belirli bir kümeden çıktı değerlerini üretir. Çıktı değerleri çözümdür.
- ❑ **Doğruluk (Correctness):** Bir algoritma, her girdi değeri kümesi için doğru çıktı değerlerini üretmelidir.
- ❑ **Sonluluk (Finiteness):** Bir algoritma, herhangi bir girdi için sınırlı sayıda adımdan sonra çıktı üretmelidir.
- ❑ **Etkililik (Effectiveness):** Algoritmanın her adımını doğru ve sınırlı bir süre içinde gerçekleştirmek mümkün olmalıdır.
- ❑ **Genellik (Generality):** Algoritma, istenen formdaki tüm problemler için çalışmalıdır.

Bir dizideki en büyük elemanı bulan algoritma

- Sözde kod:

```
procedure max( $a_1, a_2, \dots, a_n$ : integers)
  max :=  $a_1$ 
  for  $i := 2$  to  $n$ 
    if  $max < a_i$  then  $max := a_i$ 
  return max{max is the largest element}
```

- Bu algoritma önceki slayttaki tüm özellikleri sağlıyor mu?

Bazı örnek algoritma problemleri

□ Bu bölümde üç tür problem incelenecektir.

1. *Arama (Searching)* : Bir elemanın arama uzayındaki yerinin bulunması
2. *Sıralama (Sorting)* : Bir listenin elemanlarını artan /azalan sıraya koymak.
3. *Eniyileme (Optimization)* : Tüm olası girdiler üzerinden belirli bir büyüklüğün en iyi değerinin (maksimum/minimum) belirlenmesi. (Düşük maliyet, yüksek verim)

Arama (Searching)

Tanım: Genel arama problemi, ayrı elemanlar ($a_1, a_2, \dots, a_n$) listesinde bir x elemanını bulmak veya listede olmadığını belirlemektir.

- ❑ Bir arama probleminin çözümü, listedeki terimin x 'e eşit olan konumudur ($x = a_i$) veya x listede yoksa 0'dır.
- ❑ Arama sadece bir sayı dizisinde eleman aramak demek değildir!
- ❑ İki farklı arama algoritmasını inceleyeceğiz;
 - doğrusal arama
 - ikili arama.

→ Brute force

Doğrusal Arama (Linear Search)

- Doğrusal Arama algoritması, aranan sayıyı dizinin elemanlarını baştan başlayarak birer birer incelemesidir

```
procedure linear search( $x$ :integer,  
                         $a_1, a_2, \dots, a_n$ : distinct integers)  
 $i := 1$   
while ( $i \leq n$  and  $x \neq a_i$ )  
     $i := i + 1$   
if  $i \leq n$  then  $location := i$   
else  $location := 0$   
  
return  $location$  { $location$  is the subscript of the term  
                  that equals  $x$ , or is 0 if  $x$  is not found}
```

İkili Arama (Binary Search)

- İkili arama algoritması, arama uzayındaki orta noktayı dikkate alarak arama gerçekleştiren algoritmadır

procedure binary search(x : integer, $a_1, a_2, \dots, a_n$: increasing integers)

$i := 1$ { i is the left endpoint of interval}

$j := n$ { j is right endpoint of interval}

while $i < j$

$m := \lfloor (i + j) / 2 \rfloor$

if $x > a_m$ **then** $i := m + 1$

else $j := m$

if $x = a_i$ **then** $location := i$

else $location := 0$

return $location$ {location is the subscript i of the term a_i equal to x ,
or 0 if x is not found}

Sıralama (Sorting)

- ❑ Bir dizinin öğelerini sıralamak, onları artan sıraya koymaktır (sayısal sıra, alfabetik vb.).
- ❑ Sıralama önemli bir problemdir:
 - Donanım kaynaklarının önemsiz bir yüzdesi, farklı türdeki dizileri, özellikle belirli bir sırayla sunulması gereken büyük bilgi veri tabanlarını içeren uygulamaları (örneğin, müşteriye göre, parça numarası vb.) sıralamak için ayrılmıştır.
 - Çok fazla sayıda Sıralama algoritması geliştirilmiştir. Göreceli avantajları ve dezavantajları kapsamlı bir şekilde incelenmiştir.

Kabarcık (Bubble) Sort

- Kabarcık sıralama, bir dizide birden çok geçiş yapar. Sıra dışı olduğu tespit edilen her öge çifti değiştirilir.

procedure *bubblesort*($a_1, \dots, a_n$: real numbers with $n \geq 2$)

for $i := 1$ to $n - 1$

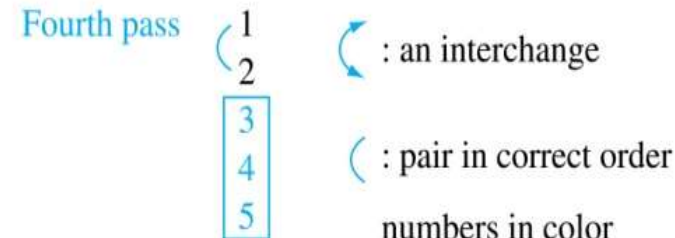
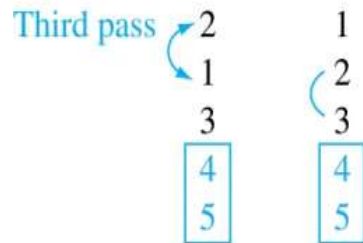
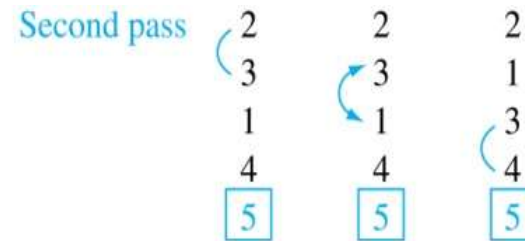
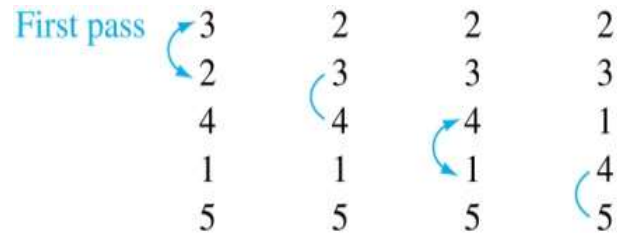
for $j := 1$ to $n - i$

if $a_j > a_{j+1}$ **then** interchange a_j and a_{j+1}

$\{a_1, \dots, a_n$ is now in increasing order $\}$

Bubble Sort

Örnek: 3 2 4 1 5 ile kabarcıklı sıralama adımlarını gösterin



: an interchange

: pair in correct order

numbers in color

guaranteed to be in correct order

- ❑ İlk geçişte en büyük eleman doğru konuma getirildi
- ❑ İkinci geçişin sonunda 2. en büyük eleman doğru konuma getirildi
- ❑ Sonraki her geçişte, doğru konuma ek bir eleman yerleştirilir

Araya Ekleme (Insertion) Sort

- ❑ Araya Ekleme sıralaması, 2. eleman ile başlar. İkinci elemanı 1. ile karşılaştırır ve daha büyük değilse ilkinden önce koyar.

- ❖ Daha sonra 3. eleman, ilk 3 eleman arasında doğru konuma getirilir.
- ❖ Sonraki her geçişte, $n + 1$. eleman ilk $n + 1$ elemanlar arasında doğru konumuna yerleştirilir.
- ❖ Doğru pozisyonu bulmak için doğrusal arama kullanılır.

```
procedure insertion sort ( $a_1, \dots, a_n$ :  
    real numbers with  $n \geq 2$ )  
    for  $j := 2$  to  $n$   
         $i := 1$   
        while  $a_j > a_i$   
             $i := i + 1$   
         $m := a_j$   
        for  $k := 0$  to  $j - i - 1$   
             $a_{j-k} := a_{j-k-1}$   
         $a_i := m$   
    {Now  $a_1, \dots, a_n$  is in increasing order}
```

Insertion Sort

Örnek: Verilen 3 2 4 1 5 diziyi sıralayınız

- A. 2 3 4 1 5 (*ilk iki pozisyon değiştirilir*)
- B. 2 3 4 1 5 (*üçüncü eleman yerinde kalır*)
- C. 1 2 3 4 5 (*dördüncüsü başlangıca yerleştirilir*)
- D. 1 2 3 4 5 (*beşinci eleman yerinde kalır*)

14.10.2023
14.10.2023
14.10.2023

Euclid's Algorithm

obeb'de bulmak için küçük sayıya gidiceğine göre kuantadan itibaren ilk bulduğumuz değer ile bit

Problem: $\gcd(m, n)$, the greatest common divisor : m ve n sayısının OBEB'i

Örnek: $\gcd(60, 24) = 12$, $\gcd(60, 0) = 60$,

Euclidean Algoritması

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

yukarıdaki eşitlik ikinci sayı (n) 0 olana kadar devam eder

Örnek:

$$\gcd(60, 24) = \gcd(24, 12) = \gcd(12, 0) = 12$$

Euclid's Algoritması Sözde Kodu

1. If $n = 0$, return m and stop; otherwise go to 2
2. Divide m by n and assign the value of the remainder to r
3. Assign the value of n to m and the value of r to n . Go to 1.

while $n \neq 0$ do

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

return m

Diğer çözüm $\gcd(m,n)$

Ardışık tamsayı kontrol algoritması

1. İki sayıdan küçük olanı $\{m, n\}$, T değişkenine atayın
2. m 'yi T 'ye bölün. Kalan 0 ise (3)'e gidin; değilse (4)'e gidin
3. n 'yi T 'ye bölün. Kalan 0 ise, T 'yi döndürün ve durun; değilse (4)'e gidin
4. T 'yi 1 azaltın ve (2) gidin

Algoritma tasarımı yöntemleri

- ❑ Brute force
- ❑ Divide and conquer
- ❑ Decrease and conquer
- ❑ Transform and conquer
- ❑ Space and time tradeoffs
- ❑ Greedy
- ❑ Dynamic programming
- ❑ Iterative improvement
- ❑ Backtracking
- ❑ Branch and bound

→ Algoritma Analizi

Bazı önemli problemler

- ❑ Sıralama - Sorting
- ❑ Arama - Searching
- ❑ Metin işleme - String processing
- ❑ Çizge problemleri - Graph problems
- ❑ Kombinasyonel problemler- Combinatorial problems
- ❑ Geometrik problemler- Geometric problems
- ❑ Sayısal problemler- Numerical problems

regula false
Bisection

Temel Veri Yapıları

Bellegin kendisi 1 boyutlu

□ Listeler

- Dizi -Array → 1 boyutlu
- Bağlantılı listeler -linked list
- String

□ Yığın - Stack

□ Kuyruk - Queue

□ Priority queue

□ Çizge - Graf

□ Ağaçlar

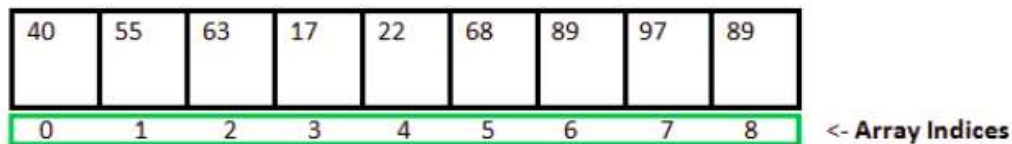
□ Kümeler

Abstraction

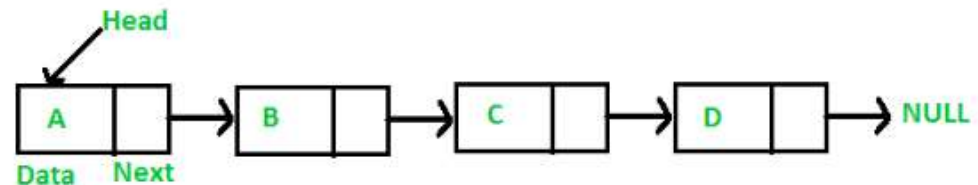
Listeler

❑ Dizi-Array / Bağlı Liste- Linked List

- Diziler belleğe ardışıl olarak yerleştirilir
- Bağlı listeler, belleğin ardışıl olmayan hücrelerine yerleştirilir
- Birden fazla elemanın saklanması işlenmesi durumunda kullanılır
- Bağlı listeler (Linked list) sıralı erişim, diziler rastgele veya sıralı erişime uygundur



Array Length = 9
First Index = 0
Last Index = 8



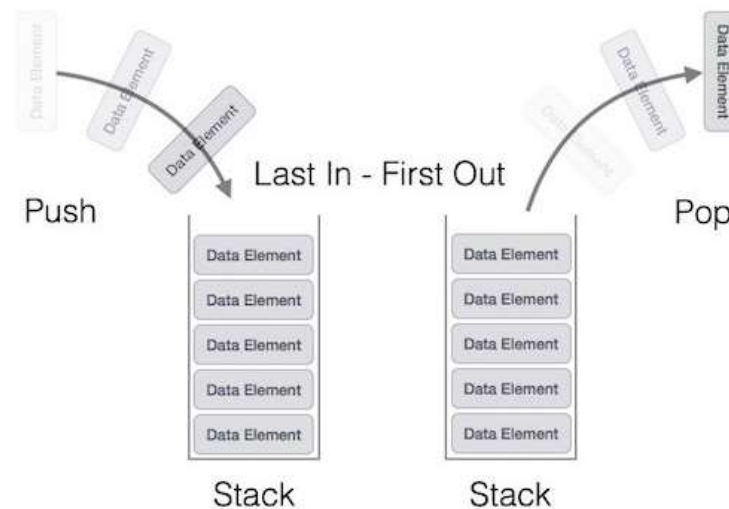
Kuyruk - Queue

- ❑ Özel amaçlı kullanılan listelerdir
- ❑ İlk gelen ilk çıkar / Son gelen son çıkar prensibine göre erişim sağlanır
(FIFO / LILO → First In - First Out / Last In - Last Out)
- ❑ Dizi veya bağlı listeler ile oluşturulabilir
- ❑ İki temel işlem yapılır. Ekle (In-Enqueue), Çıkar (Out-Dequeue)



Yığın - Stack

- ❑ Özel amaçlı kullanılan listelerdir
- ❑ İlk gelen son çıkar / Son gelen ilk çıkar prensibine göre erişim sağlanır (FILO / LIFO → First In-Last Out / Last In-First Out)
- ❑ Dizi veya bağlı listeler ile oluşturulabilir
- ❑ Arama algoritmalarında, matematiksel ifadelerin dönüşümünde kullanılır
- ❑ İki temel işlem yapılır. Ekle (push), çıkar (pop)



Yığın Uygulamaları

- ❑ İşlemci içinde SP (Stack Pointer) başta olmak üzere çok çeşitli uygulamalar vardır. Ders kapsamında aşağıdaki iki uygulamaya ağırlık verilecektir.
 - Matematiksel ifadelerin birbirine dönüşümü
 - Matematiksel ifadelerin işlem önceliğine uygun olarak değerlendirilmesi

Matematiksel İfadelerin Yazılımı

- Bir matematiksel ifade de OPERATÖR ve OPERAND'ların yazılım sırası farklı notasyonlara göre yapılabilir
- Sıralama farklı olsa da, sonuç aynıdır
- Derleyicilerin operatör önceliği vb. konularda bu notasyonların kullanımı önemlidir

Örnek: Matematiksel ifademiz $(A + B) * C$ olsun.

Operandlar: A, B ve C

Operatörler: + ve *

Yığın Uygulamaları

- ❑ **Infix notasyonu:** Operatör, iki operandın ortasına bulunur

Örnek: $A + B$, $7 - 9$

- ❑ **Prefix notasyonu:** Operatör iki operandın önünde bulunur PN (Polish Notation)

Örnek: $+ A B$, $- 7 9$

- ❑ **Postfix notasyonu:** Operatör iki operandın sonunda bulunur (Reverse Polish Notation)

Örnek: $A B +$, $7 9 -$

- ❑ Bilgisayar sistemlerinde infix yazılımdaki operatör önceliğinin çözümlenmesinin zorluğu sebebiyle postfix /prefix notasyonu dönüşümü ile işlemlerin sonucu hesaplanır!
- ❑ Bu dönüşümler, **infix** $\leftrightarrow$ **postfix**, **infix** $\leftrightarrow$ **prefix**, **prefix** $\leftrightarrow$ **postfix** olabilir
- ❑ Verilen bir notasyonun sayısal sonucunun hesaplanması da çok sık karşılaşılan uygulamalardandır
- ❑ Dönüşümlerde Stack / İkili Ağaçlar kullanılır

Infix ifadenin değeriendirilmesi

- Operands are real numbers.
- Permitted operators: +, -, *, /, ^ (exponentiation)
- Blanks are permitted in expression.
- Parenthesis are permitted

Example:

$A * (B + C) / D$

$2 * (5 + 3) / 4$

Output: 4

Approach: Use Stacks

We will use two stacks

- Operand stack: This stack will be used to keep track of numbers.
- Operator stack: This stack will be used to keep operations (+, -, *, /, ^)

Order of precedence of operations–

1. ^ (Exponential)
2. / *
3. + –

Note: brackets () are used to override these rules.

Infix ifadenin değeriendirilmesi

Algorithm:

Iterate through given expression, one character at a time

1. If the character is an operand, push it to the operand stack.
2. If the character is an operator,
 1. If the operator stack is empty then push it to the operator stack.
 2. Else If the operator stack is not empty,
 - If the character's precedence is greater than or equal to the precedence of the stack top of the operator stack, then push the character to the operator stack.
 - If the character's precedence is less than the precedence of the stack top of the operator stack then do **Process** (as explained above) until character's precedence is less or stack is not empty.
3. If the character is "(", then push it onto the operator stack.
4. If the character is ")", then do **Process** (as explained above) until the corresponding "(" is encountered in operator stack. Now just pop out the "(".

Once the expression iteration is completed and the operator stack is not empty, do **Process** until the operator stack is empty. The values left in the operand stack is our final result.

Örn: Infix değerlendirme

Infix Expression: $2 * (5 * (3+6)) / 15-2$				
Token	Action	Operand Stack	Operator Stack	Notes
2	Push it to operand stack	2		
*	Push it to operator stack	2	*	
(	Push it to operator stack	2	(*	
5	Push it to operand stack	5 2	(*	
*	Push it to operator stack	5 2	(* *	
(	Push it to operator stack	5 2	(* (*	
3	Push it to operand stack	3 5 2	(* (*	
+	Push it to operator stack	3 5 2	(* (+ *	
6	Push it to operand stack	6 3 5 2	(* (+ *	
)	Pop 6 and 3 from operand stack	5 2	(* (+ *	Do process until (is popped from operator stack
	Pop + from operator stack	5 2	(* (*	
	Do $6+3 = 9$	5 2	(* (*	
	Push 9 to operand stack	9 5 2	(* (*	
	Pop (from operator stack	9 5 2	(* *	
)	Pop 9 and 5 from operand stack	2	(*	Do process until (is popped from operator stack
	Pop * from operator stack	2	(*	
	Do $9*5 = 45$	2	(*	
	Push 45 into operand stack	45 2	(*	
	Pop (from operator stack	45 2	*	
/	Push / into operator stack	45 2	/ *	/ has equal precedence to *
15	Push 15 to operand stack	15 45 2	/ *	
-	Pop 15 and 45 from operand stack	2	/ *	- has lower precedence than / , do the process
	Pop / from operator stack	2	*	
	Do $45/15 = 3$	2	*	
	Push 3 into operand stack	3 2	*	
	Pop 3 and 2 from operand stack		*	- has lower precedence than * , do the process
	Pop * from operator stack			
	Do $3*2 = 6$			
	Push 6 into operand stack	6		
	Push - into operator stack	6	-	- has equal precedence to +
2	Push 2 into operand stack	2 6	-	
	Pop 2 and 6 from the operand stack			Given expression is iterated, do Process till operator stack is not empty, It will give the final result
	Pop - from operator stack			
	Do $6-2 = 4$			
	Push 4 to operand stack	4		

Prefix ifadenin değerdendirme

Prefix: + 500 40

Output: 540

Explanation: Infix expression of the above prefix is: $500 + 40$ which resolves to 540

Prefix: - / * 20 * 50 + 3 6 300 2

Output: 28

Explanation: Infix expression of above prefix is: $20 * (50 * (3+6)) / 300 - 2$ which resolves to 28

Approach: Use Stack

Algorithm:

Reverse the given expression and Iterate through it, one character at a time

1. If the character is a digit, initialize String temp;
 - while the next character is not a digit
 - do temp = temp + currentDigit
 - convert Reverse temp into Number.
 - push Number to the stack.
2. If the character is an operator,
 - pop the operand from the stack, say it's s1.
 - pop the operand from the stack, say it's s2.
 - perform **(s1 operator s2)** and push it to stack.
3. Once the expression iteration is completed, The stack will have the final result. Pop from the stack and return the result.

Örn: Prefix ifadenin değerlendirilmesi

Prefix Expression : - / * 20 * 50 + 3 6 300 2		
Reversed Prefix Expression: 2 003 6 3 + 05 * 02 * / -		
Token	Action	Stack
2	Reverse 2, Push 2 to stack	[2]
003	Reverse 003, Push 300 to stack	[2, 300]
6	Reverse 6, Push 6 to stack	[2, 300, 6]
3	Reverse 3, Push 3 to stack	[2, 300, 6, 3]
+	Pop 3 from stack	[2, 300, 6]
	Pop 6 from stack	[2, 300]
	Push 3+6 =9 to stack	[2, 300, 9]
05	Reverse 05, Push 50 to stack	[2, 300, 9, 50]
*	Pop 50 from stack	[2, 300, 9]
	Pop 9 from stack	[2, 300]
	Push 50*9=450 to stack	[2, 300, 450]
02	Reverse 02, Push 20 to stack	[2, 300, 450, 20]
*	Pop 20 from stack	[2, 300, 450]
	Pop 450 from stack	[2, 300]
	Push 20*450=9000 to stack	[2, 300, 9000]
/	Pop 9000 from stack	[2, 300]
	Pop 300 from stack	[2]
	Push 9000/300=30 to stack	[2, 30]
-	Pop 30 from stack	[2]
	Pop 2 from stack	[]
	Push 30-2=28 to stack	[28]

Postfix ifadenin değeriendirilmesi

Input: Postfix expression: **A B +**

Output: Infix expression- **(A + B)**

Input: Postfix expression: **ABC/-AK/L-***

Output: Infix expression: **((A-(B/C))*((A/K)-L))**

Algorithm:

Iterate through given expression, one character at a time

1. If the character is an operand, push it to the operand stack.
2. If the character is an operator,
 1. pop an operand from the stack, say it's s1.
 2. pop an operand from the stack, say it's s2.
 3. perform **(s2 operator s1)** and push it to stack.
3. Once the expression iteration is completed, The stack will have the final result. Pop from the stack and return the result.

Örn:Postfix ifadenin değeriendirilmesi

Postfix Expression : 2536+**5/2-		
Token	Action	Stack
2	Push 2 to stack	[2]
5	Push 5 to stack	[2, 5]
3	Push 3 to stack	[2, 5, 3]
6	Push 6 to stack	[2, 5, 3, 6]
+	Pop 6 from stack	[2, 5, 3]
	Pop 3 from stack	[2, 5]
	Push 3+6 =9 to stack	[2, 5, 9]
*	Pop 9 from stack	[2, 5]
	Pop 5 from stack	[2]
	Push 5*9=45 to stack	[2, 45]
*	Pop 45 from stack	[2]
	Pop 2 from stack	[]
	Push 2*45=90 to stack	[90]
5	Push 5 to stack	[90, 5]
/	Pop 5 from stack	[90]
	Pop 90 from stack	[]
	Push 90/5=18 to stack	[18]
2	Push 2 to stack	[18, 2]
-	Pop 2 from stack	[18]
	Pop 18 from stack	[]
	Push 18-2=16 to stack	[16]
Result : 16		

Infix-Postfix Dönüşümü

Input: Infix expression - $A + B$
Output: Postfix expression- $AB+$

Input: Infix expression - $A+B*(C^D-E)$
Output: Postfix expression- $ABCD^E-*+$

Approach: Use Stacks

- **Operator stack:** This stack will be used to keep operations (+, -, *, /, ^)

Order of precedence of operations–

1. ^ (Exponential)
2. / *
3. + –

Note: brackets () are used to override these rules.

Algorithm:

Initialize result as a blank string, Iterate through given expression, one character at a time

1. If the character is an operand, add it to the result.
2. If the character is an operator.
 - If the operator stack is empty then push it to the operator stack.
 - Else If the operator stack is not empty,
 - If the operator's precedence is greater than or equal to the precedence of the stack top of the operator stack, then push the character to the operator stack.
 - If the operator's precedence is less than the precedence of the stack top of operator stack then *"pop out an operator from the stack and add it to the result until the stack is empty or operator's precedence is greater than or equal to the precedence of the stack top of operator stack"*. then push the operator to stack.
3. If the character is "(", then push it onto the operator stack.
4. If the character is ")", then *"pop out an operator from the stack and add it to the result until the corresponding "(" is encountered in operator stack. Now just pop out the "("*.

Once the expression iteration is completed and the operator stack is not empty, *"pop out an operator from the stack and add it to the result"* until the operator stack is empty. The result will be our answer, postfix expression.

Örn: Infix-Postfix Dönüşümü

Infix Expression : A+B*(C^D-E)				
Token	Action	Result	Stack	Notes
A	Add A to the result	A		
+	Push + to stack	A	+	
B	Add B to the result	AB	+	
*	Push * to stack	AB	* +	* has higher precedence than +
(	Push (to stack	AB	{ * +	
C	Add C to the result	ABC	{ * +	
^	Push ^ to stack	ABC	^ { * +	
D	Add D to the result	ABCD	^ { * +	
-	Pop ^ from stack and add to result	ABCD^	{ * +	- has lower precedence than ^
	Push - to stack	ABCD^	- { * +	
E	Add E to the result	ABCD^E	- { * +	
)	Pop - from stack and add to result	ABCD^E-	{ * +	Do process until (is popped from stack
	Pop { from stack	ABCD^E-	* +	
	Pop * from stack and add to result	ABCD^E-*	+	Given expression is iterated, do Process till stack is not Empty, It will give the final result
	Pop + from stack and add to result	ABCD^E-*+		
Postfix Expression : ABCD^E-*+				

$ABCD^E-$
 $*$
 $+$

Infix-Prefix Dönüşümü

Input: Infix expression - **A + B**
Output: Prefix expression- **+AB**

Input: Infix expression - **A+B*(C^D-E)**
Output: Prefix expression- **+A*B-^CDE**

Approach: Use Stack

- Operator stack: This stack will be used to keep operations (+, -, *, /, ^)

Order of precedence of operations–

1. ^ (Exponential)
2. / *
3. + –

Note: brackets () are used to override these rules.

Algorithm:

- Reverse the given infix expression. (Note: do another reversal only for brackets).
- Do **Infix to postfix expression** and get the result.
- Reverse the result to get the final expression. (prefix expression) .

Infix – Prefix dönüşümü Stack ile doğrudan yapılmaz, önce ifade tersten yazılıp Infix – Postfix dönüşümü ile dolaylı olarak yapılır.

Örn: Infix-Prefix Dönüşümü

Infix Expression : $A+B*(C^D-E)$				
Reverse Infix expression: $)E-D^C(*B+A$				
Reverse brackets: $(E-D^C)*B+A$				
Token	Action	Result	Stack	Notes
(	Push (to stack		(	
E	Add E to the result	E	(	
-	Push - to stack	E	(-	
D	Add D to the result	ED	(-	
^	Push ^ to stack	ED	(- ^	
C	Add C to the result	EDC	(- ^	
)	Pop ^ from stack and add to result	EDC^	(-	Do process until (is popped from stack
	Pop - from stack and add to result	EDC^-	(	
	Pop (from stack	EDC^-		
*	Push * to stack	EDC^-	*	
B	Add B to the result	EDC^-B	*	
+	Pop * from stack and add to result	EDC^-B		- has lower precedence than ^
	Push + to stack	EDC^-B*	+	
A	Add A to the result	EDC^-B*A	+	
	Pop + from stack and add to result	EDC^-B*A+		Given expression is iterated, do Process till stack is not Empty, It will give the final result
Prefix Expression (Reverse Result): $+A*B^CDE$				

EDC^-B*A+

+

Postfix-Infix Dönüşümü

Input: Postfix expression: **A B +**

Output: Infix expression- **(A + B)**

Input: Postfix expression: **ABC/-AK/L-***

Output: Infix expression: **((A-(B/C))*((A/K)-L))**

Algorithm:

Iterate the given expression from left to right, one character at a time

1. If a character is operand, push it to stack.
2. If a character is an operator,
 1. pop operand from the stack, say it's s1.
 2. pop operand from the stack, say it's s2.
 3. perform **(s2 operator s1)** and push it to stack.
3. Once the expression iteration is completed, initialize the result string and pop out from the stack and add it to the result.
4. Return the result.

Örn: Postfix-Infix Dönüşümü

Postfix Expression : ABC/-AK/L-*			
Token	Action	Stack	Notes
A	Push A to stack	[A]	
B	Push B to stack	[A, B]	
C	Push C to stack	[A, B, C]	
/	Pop C from stack	[A, B]	Pop two operands from stack, C and B. Perform B/C and push (B/C) to stack
	Pop B from stack	[A]	
	Push (B/C) to stack	[A, (B/C)]	
-	Pop (B/C) from stack	[A]	Pop two operands from stack, (B/C) and A. Perform A-(B/C) and push (A-(B/C)) to stack
	Pop A from stack	[]	
	Push (A-(B/C)) to stack	[[A-(B/C)]]	
A	Push A to stack	[[A-(B/C)), A]	
K	Push K to stack	[[A-(B/C)), A, K]	
/	Pop K from stack	[[A-(B/C)), A]	Pop two operands from stack, K and A. Perform A/K and push (A/K) to stack
	Pop A from stack	[[A-(B/C)]]	
	Push (A/K) to stack	[[A-(B/C)), (A/K)]	
L	Push L to stack	[[A-(B/C)), (A/K), L]	
-	Pop L from stack	[[A-(B/C)), (A/K)]	Pop two operands from stack, L and (A/K). Perform (A/K)-L and push ((A/K)-L) to stack
	Pop (A/K) from stack	[[A-(B/C)]]	
	Push ((A/K)-L) to stack	[[A-(B/C)), ((A/K)-L)]	
*	Pop ((A/K)-L) from stack	[[A-(B/C)]]	Pop two operands from stack, (A/K)-L and A-(B/C). Perform (A-(B/C))*((A/K)-L) and push ((A-(B/C))*((A/K)-L)) to stack
	Pop (A-(B/C)) from stack	[]	
	Push ((A-(B/C))*((A/K)-L)) to stack	[[A-(B/C))*((A/K)-L)]]	
Infix Expression: ((A-(B/C))*((A/K)-L))			

Prefix-Infix Dönüşümü



Example:

Input: Prefix expression: + A B
Output: Infix expression- (A + B)

Input: Prefix expression: *-A/BC-/AKL
Output: Infix expression: ((A-(B/C))*((A/K)-L))

Approach: Use *Stacks*

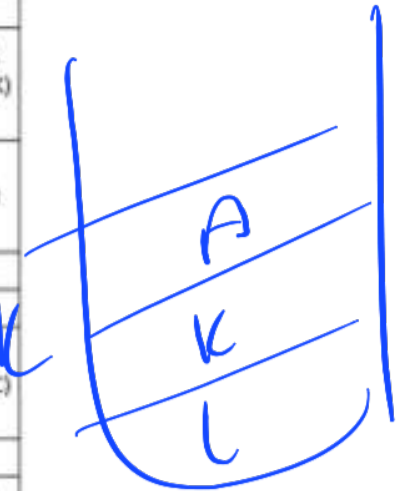
Algorithm:

Iterate the given expression from right to left (in reverse order), one character at a time

1. If character is operand, push it to stack.
2. If character is operator,
 1. pop operand from stack, say it's s1.
 2. pop operand from stack, say it's s2.
 3. perform (s1 operator s2) and push it to stack.
3. Once the expression iteration is completed, initialize result string and pop out from stack and add it to result.
4. Return the result.

Örn: Prefix-Infix Dönüşümü

Prefix Expression : *-A/BC-/AKL			
Iterate right to left			
Token	Action	Stack	Notes
L	Push L to stack	[L]	
K	Push K to stack	[L, K]	
A	Push A to stack	[L, K, A]	
/	Pop A from stack	[L, K]	Pop two operands from stack, A and K. Perform A/K and push (A/K) to stack
	Pop K from stack	[L]	
	Push (A/K) to stack	[L, (A/K)]	
-	Pop (A/K) from stack	[L]	Pop two operands from stack, (A/K) and L. Perform (A/K)-L and push ((A/K)-L) to stack
	Pop L from stack	[]	
	Push ((A/K)-L) to stack	[((A/K)-L)]	
C	Push C to stack	[((A/K)-L), C]	
B	Push B to stack	[((A/K)-L), C, B]	
/	Pop B from stack	[((A/K)-L), C]	Pop two operands from stack, B and C. Perform B/C and push (B/C) to stack
	Pop C from stack	[((A/K)-L)]	
	Push (B/C) to stack	[((A/K)-L), (B/C)]	
A	Push A to stack	[((A/K)-L), (B/C), A]	
-	Pop A from stack	[((A/K)-L), (B/C)]	Pop two operands from stack, A and (B/C). Perform A-(B/C) and push (A-(B/C)) to stack
	Pop (B/C) from stack	[((A/K)-L)]	
	Push (A-(B/C)) to stack	[((A/K)-L), (A-(B/C))]	
*	Pop (A-(B/C)) from stack	[((A/K)-L)]	Pop two operands from stack, (A-(B/C)) and ((A/K)-L). Perform (A-(B/C))*((A/K)-L) and push ((A-(B/C))*((A/K)-L)) to stack
	Pop ((A/K)-L) from stack	[]	
	Push ((A-(B/C))*((A/K)-L)) to stack	(((A-(B/C))*((A/K)-L)))	
Infix Expression: ((A-(B/C))*((A/K)-L))			



Handwritten red text at the bottom of the slide: $((A-(B/C)) * ((A/K)-L))$

Prefix-Postfix Dönüşümü

Input: Prefix expression: + A B
Output: Postfix expression: A B +

Input: Prefix expression: *-A/BC-/AKL
Output: Postfix expression: ABC/-AK/L-*

Iterate the given expression from right to left, one character at a time

1. If the character is operand, push it to the stack.
2. If the character is operator,
 1. Pop an operand from the stack, say it's s1.
 2. Pop an operand from the stack, say it's s2.
 3. perform **(s1 s2 operator)** and push it to stack.
3. Once the expression iteration is completed, initialize the result string and pop out from the stack and add it to the result.
4. Return the result.

Please walk through the example below for more understanding.

herden baslıca
işletici
nereye koyca

Örn: Postfix-Prefix Dönüşümü

Prefix Expression : *-A/BC-/AKL

Iterate right to left

Token	Action	Stack	Notes
L	Push L to stack	[L]	
K	Push K to stack	[L, K]	
A	Push A to stack	[L, K, A]	
/	Pop A from stack	[L, K]	Pop two operands from stack, A and K. Perform AK/ and push AK/ to stack
	Pop K from stack	[L]	
	Push AK/ to stack	[L, AK/]	
-	Pop AK/ from stack	[L]	Pop two operands from stack, AK/ and L. Perform AK/L- and push AK/L- to stack
	Pop L from stack	[]	
	Push AK/L- to stack	[AK/L-]	
C	Push C to stack	[AK/L-, C]	
B	Push B to stack	[AK/L-, C, B]	
/	Pop B from stack	[AK/L-, C]	Pop two operands from stack, B and C. Perform BC/ and push BC/ to stack
	Pop C from stack	[AK/L-]	
	Push BC/ to stack	[AK/L-, BC/]	
A	Push A to stack	[AK/L-, BC/, A]	
-	Pop A from stack	[AK/L-, BC/]	Pop two operands from stack, A and BC/. Perform ABC/- and push ABC/- to stack
	Pop BC/ from stack	[AK/L-]	
	Push ABC/- to stack	[AK/L-, ABC/-]	
*	Pop ABC/- from stack	[AK/L-]	Pop two operands from stack, ABC/- and AK/L-. Perform ABC/-AK/L-* and push ABC/-AK/L-* to stack
	Pop AK/L- from stack	[]	
	Push ABC/-AK/L-* to stack	[ABC/-AK/L-*]	

Postfix Expression: ABC/-AK/L-*

Postfix-Prefix Dönüşümü

ABC/-

AK/L-

Input: Postfix expression: A B +

Output: Prefix expression: + A B

Input: Postfix expression: ABC/-AK/L-*

Output: Infix expression: *-A/BC-/AKL

Algorithm:

Iterate the given expression from left to right, one character at a time

1. If the character is operand, push it to stack.
2. If the character is operator,
 1. Pop operand from the stack, say it's s1.
 2. Pop operand from the stack, say it's s2.
 3. perform (**operator s2 s1**) and push it to stack.
3. Once the expression iteration is completed, initialize the result string and pop out from the stack and add it to the result.
4. Return the result.

Örn: ~~Prefix~~-~~Postfix~~ Dönüşümü

Postfix Expression : ABC/-AK/L.*			
Token	Action	Stack	Notes
A	Push A to stack	[A]	
B	Push B to stack	[A, B]	
C	Push C to stack	[A, B, C]	
/	Pop C from stack	[A, B]	Pop two operands from stack, C and B. Perform /BC and push /BC to stack
	Pop B from stack	[A]	
	Push /BC to stack	[A, /BC]	
-	Pop /BC from stack	[A]	Pop two operands from stack, /BC and A. Perform -A/BC and push -A/BC to stack
	Pop A from stack	[]	
	Push -A/BC to stack	[-A/BC]	
A	Push A to stack	[-A/BC, A]	
K	Push K to stack	[-A/BC, A, K]	
/	Pop K from stack	[-A/BC, A]	Pop two operands from stack, K and A. Perform /AK and push /AK to stack
	Pop A from stack	[-A/BC]	
	Push /AK to stack	[-A/BC, /AK]	
L	Push L to stack	[-A/BC, /AK, L]	
-	Pop L from stack	[-A/BC, /AK]	Pop two operands from stack, L and /AK. Perform -/AKL and push -/AKL to stack
	Pop /AK from stack	[-A/BC]	
	Push -/AKL to stack	[-A/BC, -/AKL]	
*	Pop -/AKL from stack	[-A/BC]	Pop two operands from stack, -/AKL and -A/BC. Perform *-A/BC-/AKL and push *-A/BC-/AKL to stack
	Pop -A/BC from stack	[]	
	Push *-A/BC-/AKL to stack	[-A/BC-/AKL]	
Prefix Expression: *-A/BC-/AKL			

* - A/B/C - /A K C

- /A K C

- A/B C

Algoritma Analizi



Algoritma analizi

□ Konular

- Doğruluk - correctness
- Zaman - time efficiency
- Bellek - space efficiency
- En iyi çözüm - optimality

□ Yaklaşımlar

- Teorik analiz - theoretical analysis
- Kaba analiz - empirical analysis (sayaç/süre ölçmek gibi)

Teorik Analiz

Zaman verimliliği, girdi boyutunun bir fonksiyonu olarak temel işlemin tekrar sayısı belirlenerek analiz edilir.

Temel İşlem (Basic operation): Algoritmanın çalışma süresini en çok etkileyen işlemdir. Sıralamada temel işlemdir. ??

Kaba Analiz

- Bir giriş verisi seçilir
- Zaman birimini seç (milisaniye)
veya
Yürütülen temel işlem adımlarının sayısı
- Deneysel veriler ile analiz yapılır

Problemler - Temel işlemler

<i>Problem</i>	<i>Giriş verisinin büyüklüğü</i>	<i>Temel işlemler</i>
n elemanlı bir dizide eleman arama	Dizinin eleman sayısı. n	Elemanları karşılaştırma
İki matrisin çarpımı	Matris boyutu veya toplam eleman sayısı	İki sayının çarpımı
İnteger bir sayının asal olup olmadığının kontrolü	N sayısının dijital sayısı (ikili gösterilim)	Bölme
Graf problemi	Düğüm / Kenar sayısı	Düğüm ve kenarların gezilmesi

Temel Kavramlar

Yürütme Zamanı (Running Time) $T(n)$

- Algoritmanın işlevini yerine getirebilmesi için kabul edilen işlemlerden kaç adet yürütülmesi gerektiğini gösteren matematiksel bir ifadedir
- İşlem olarak karşılaştırma sayısı, çevrim sayısı, aritmetik işlem sayısı kabul edilir
- $T(n)$ hesaplanırken hangi işleme göre hesaplandığı bildirilmelidir

Alan Maliyeti (Space Cost)

- Algoritmanın işlevini yerine getirebilmesi için gerekli olan bellek ihtiyacıdır
- Maliyet programın kodu, veri yapısı alanları ve yığın bellek alanıdır
- Alan maliyeti algoritma rekürsif değilse ve veri yapısı alanı çok çok büyük değilse pek hesaplanmaz

Asimptotik İfade

- Bir problemin büyümesinin üst, alt ve ortalama sınırını belirleyen ifadedir
- Big O $\rightarrow O$, Big Omega $\rightarrow \Omega$, Big Theta $\rightarrow \theta$

Alan Karmaşıklığı (Space Complexity)

- Eleman sayısı n 'nin büyük değerleri için bellek alanı gereksiniminin artışını gösteren asimptotik bir ifadedir

Zaman Karmaşıklığı (Time Complexity)

- Algoritmanın yürütülürken geçecek zamanın artması hakkında bilgi veren asimptotik bir ifadedir
- $O(g(n))$, $\theta(g(n))$, $\Omega(g(n))$, $o(g(n))$ gösterilir
- Veri kümesinin büyümesi durumunda çalışma süresinin nasıl etkileneceği hakkında bilgi verir

Yürütme Zamanı-Running Time $T(n)$

$T(n) = 2n^2 - 2n + 5$ olabilir.

n , ifadenin bağımsız değişkenidir ve temel işlem sayısına bağlıdır.

$T(n) = 1$	sabit
$T(n) = \log n$	logaritmik
$T(n) = n$	lineer
$T(n) = n^2$	karesel

Handwritten red notes at the bottom of the slide:

- On the left, a vertical list of numbers: 0, 1, 2, 3.
- On the right, a vertical list of numbers: 1, 0, 1, 2.

Örnek

```
float BulOrta (float A[], int n)
```

```
{
```

```
    float ortalama, toplam=0;  $\longrightarrow$  toplam=0 1 işlem
```

```
    int k;
```

```
    for (k=0; k<n; k++)
```

```
        toplam=toplam + A[k];  $\longrightarrow$ 
```

```
    ortalama = toplam/n;
```

```
    return ortalama;
```

```
}
```

1 işlem

Döngü dışında bölme ve atama **2 işlem**

k=0 **1 kere**

k<n **(n+1) kere**

k++ **n kere**

} 1+(n+1)+n=2n+2

} atama ve toplama 2 işlem
döngü içerisinde n kere } 2n

$$T(n) = 1 + 2n + 2 + 2n + 2 + 1 = 4n + 6$$

$$T(n) = 4n + 6$$

Örnek

```
float BulEnkucuk (float A[ ])
```

```
{
```

```
    float enkucuk;
```

```
    int k;
```

```
    enkucuk=A[0];
```

atama 1 işlem

```
    for (k=1; k<n; k++)
```

k=1 1 kere

k<n n kere

k++ n-1 kere

1+n+(n-1)=2n

```
        if (A[k] < enkucuk)
```

karşılaştırma 1 işlem n-1 kere

```
        enkucuk=A[k];
```

```
    return enkucuk;
```

1 işlem

Bu işlemin kaç kez yürütüleceği belli değil,
en kötü durumda n-1 kere

```
}
```

$$T(n)=1+2n+(n-1)+(n-1)+1=4n$$

$$T(n) = 4n$$

i < 2 n=3

0
1
2

```

public static int [] selectionsort(int [] A,int n)
{
    int tmp;
    int min;

    for(int i=0; i < n-1; i++) I
    {
        min=i; II
        for(int j=i; j < n; j++) III
        {
            if (A[j] < A[min]){ IV
                min=j; V
            }
        }
        tmp=A[i]; VI
        A[i]=A[min];
        A[min]=tmp;
    }
    return A; VII
}

```

	İşlem	Tekrar	Toplam
I	1,1,1	1,n,n	2n+1
II	1	n	n
III	1,1,1	n, n(n+1)/2, n(n+1)/2	n+2n(n+1)/2
IV	1	n(n+1)/2	n(n+1)/2
V	1	n(n+1)/2	n(n+1)/2
VI	1,1,1	n,n,n	3n
VII	1	1	1
		4n(n+1)/2+7n+2	

$$T(n) = 2n^2 + 9n + 2$$

Matris Toplamı

for (i=0; i<n; i++) $\longrightarrow 1+(n+1) + \underline{n} = 2n + 2$
for (j=0; j<m; j++) $\longrightarrow (1+(m+1) + \underline{m}) * \underline{n} = (2m + 2)n$
C[i][j] = A[i][j] + B[i][j]; $\longrightarrow (2 * m * n)$

$$4mn + 4n + 2$$

2 $2 \times m \times n$

m ve n eşit alırsak $T(n) = 4n^2 + 4n + 2$

Fonksiyonların Büyümesi ve Asimptotik Notasyonlar

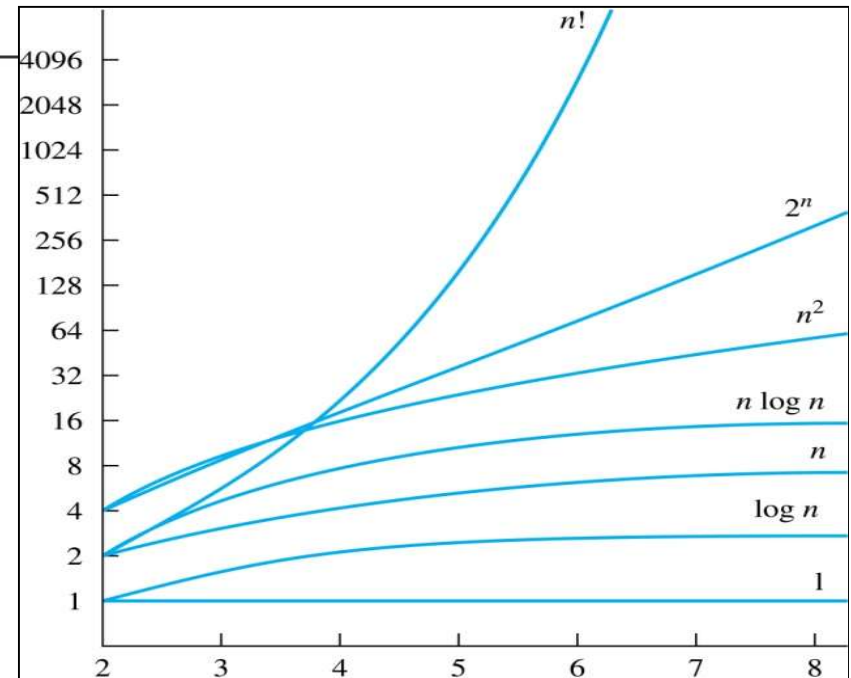
- Hem bilgisayar biliminde hem de matematikte, bir fonksiyonun ne kadar hızlı büyüdüğü sıklıkla incelenir
- Bilgisayar biliminde girdinin boyutu büyüdükçe, algoritmanın bir problemi ne kadar hızlı çözebileceğini bilmek isteriz
 - Aynı problemi çözmek için iki farklı algoritmanın verimliliğini karşılaştırabiliriz
 - Girdi büyüdükçe belirli bir algoritmayı kullanmanın pratik olup olmadığını da belirleyebiliriz

Karmaşıklık

- ❑ Bir algoritmanın, üzerinde işlem yapacağı veri kümesinin eleman sayısının artması durumunda yürütme zaman maliyetinin nasıl değişeceğini gösteren asimptotik bir ifadedir
 - Yürütme maliyetine, zaman karmaşıklığı
 - Bellek kullanım maliyetine, alan karmaşıklığı
- ❑ Karmaşıklıkta ki amaç gerçek zaman ve bellek büyüklüğü bilgisi değil, veri kümesi büyüdüğünde maliyet bilgisinin değişimidir

$n \rightarrow \infty$ bazı önemli fonksiyonların büyüme hızı

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

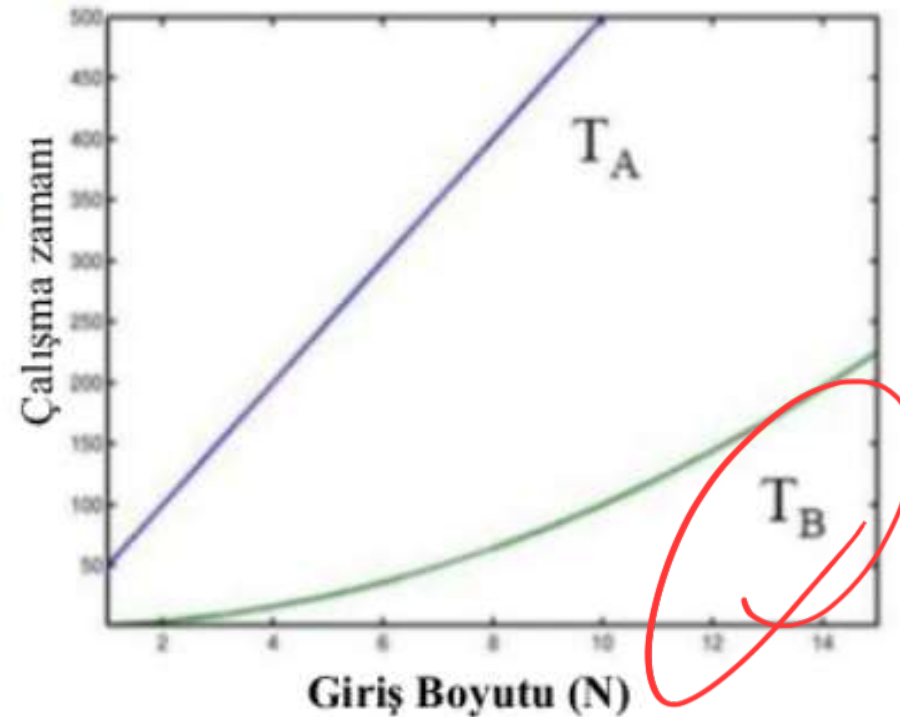


Algoritma Analiz Türleri

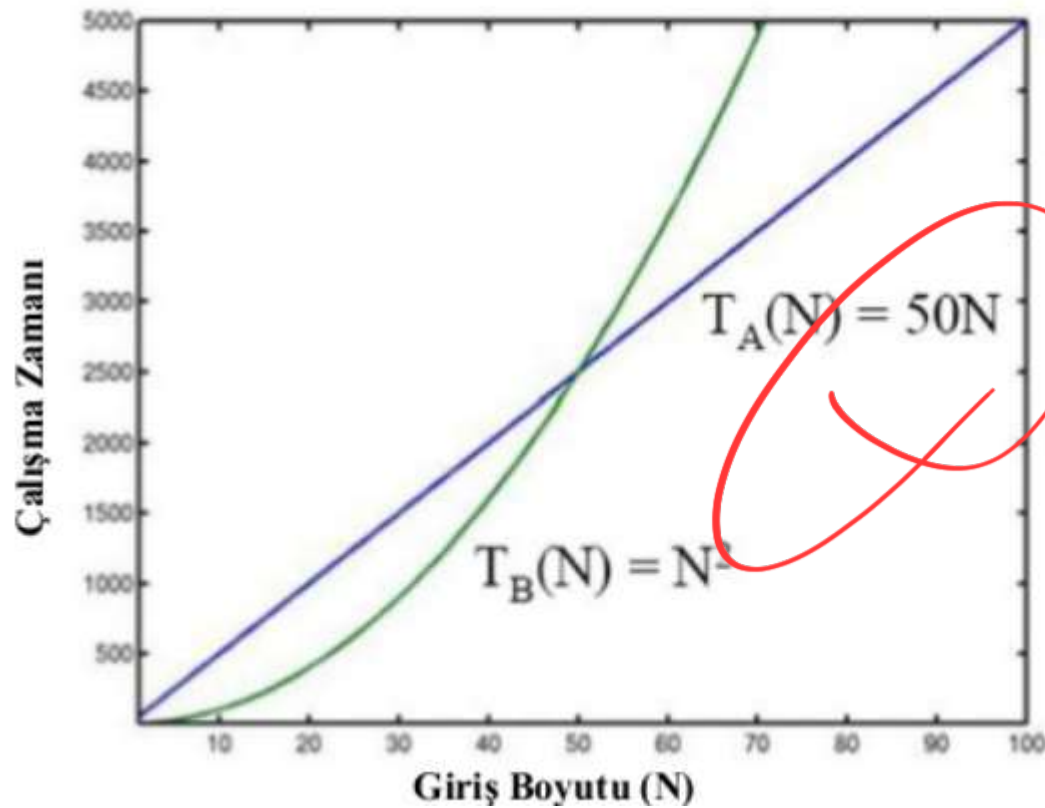
- **Worst case (en kötü):** Algoritma çalışmasının en fazla sürede gerçekleştiği analiz türüdür. En kötü durum, çalışma zamanında bir üst sınırdır ve o algoritma için verilen durumdan *“daha uzun sürmeyeceği”* **garantisi** verir. Bazı algoritmalar için en kötü durum *oldukça sık rastlanır*. Arama algoritmasında, aranan öge genellikle **dizide olmaz** dolayısıyla **döngü N kez çalışır**.
- **Best case (en iyi):** Algoritmanın en kısa sürede ve en az adımda çalıştığı giriş durumu olan analiz türüdür. Çalışma zamanında bir alt sınırdır.
- **Average case (ortalama):** Algoritmanın ortalama sürede ve ortalama adımda çalıştığı giriş durumu olan analiz türüdür.

- Bir problemi çözmek için **A ve B** şeklinde iki algoritma verildiğini düşünelim.
- Giriş boyutu **N** için aşağıda A ve B algoritmalarının çalışma zamanı **T_A** ve **T_B** fonksiyonları verilmiştir.

Hangi algoritmayı seçersiniz?



- N büyüdüğü zaman A ve B nin çalışma zamanı:



Şimdi hangi
algoritmayı
seçersiniz?

- Asimptotik notasyon, **eleman sayısı n'nin sonsuza gitmesi durumunda** algoritmanın, benzer işi yapan **algoritmalarla karşılaştırmak** için kullanılır.
- Eleman sayısının *küçük olduğu durumlar mümkün olabilir* fakat bu *birçok uygulama için geçerli değildir*.
- Verilen iki algoritmanın çalışma zamanını $T_1(N)$ ve $T_2(N)$ fonksiyonları şeklinde gösterilir. Hangisinin **daha iyi** olduğunu belirlemek için bir yol belirlememiz gerekiyor.
 - Big-O (Big O): Asimptotik üst sınır *upper bound*
 - Big Ω (Big Omega): Asimptotik alt sınır *worst case*
 - Big Θ (Big Teta): Asimptotik alt ve üst sınır *Best case*

$f(x)$, bir algoritmanın fonksiyon şeklindeki gösterimi ise karmaşıklık $O(f(x))$, $\Omega(f(x))$, ... şeklinde gösterilir.

	n=1	n=2	n=4	n=8	n=16	n=32
1	1	1	1	1	1	1
$\log n$	0	1	2	3	4	5
n	1	2	4	8	16	32
$n \log n$	0	2	8	24	64	160
n^2	1	4	16	64	256	1024
n^3	1	8	64	512	4096	32768
2^n	2	4	16	256	65536	4294967296
$n!$	1	2	24	40320	20.9T	Don't ask!

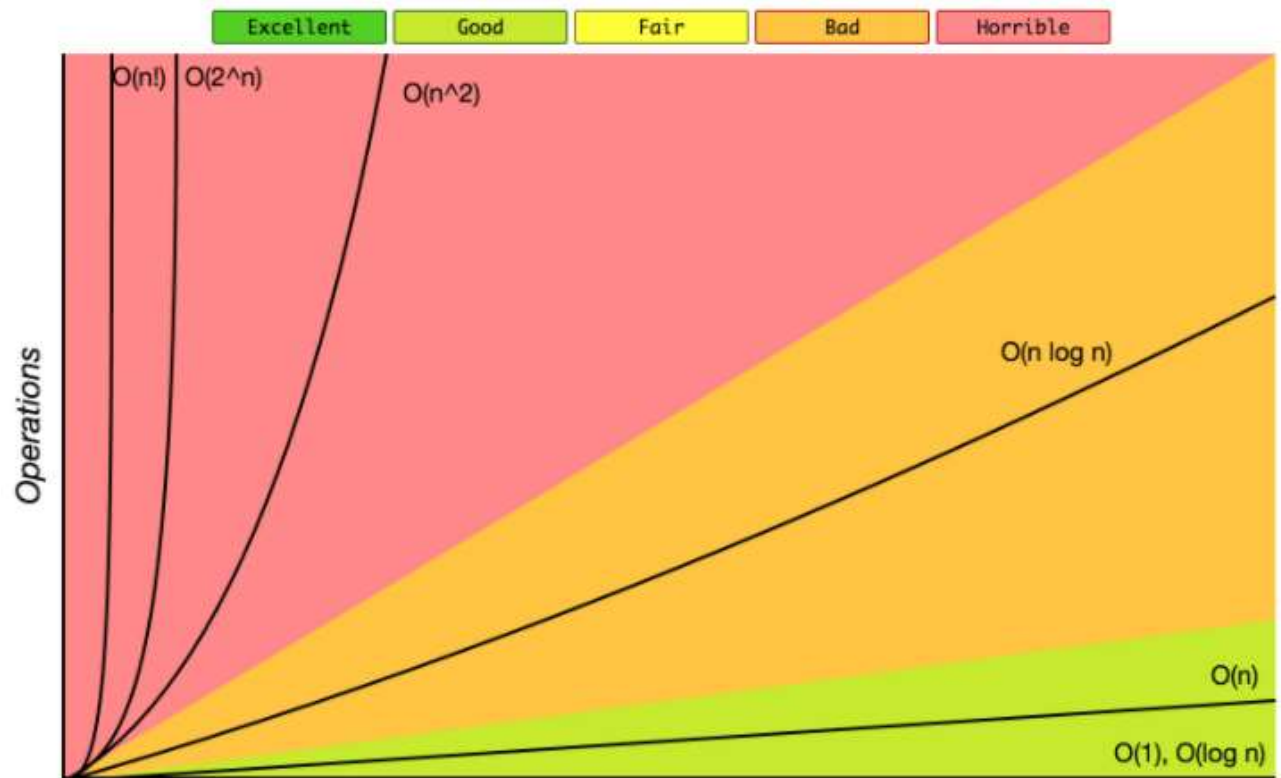


TABLE 1 Commonly Used Terminology for the Complexity of Algorithms.

<i>Complexity</i>	<i>Terminology</i>
$\Theta(1)$	Constant complexity
$\Theta(\log n)$	Logarithmic complexity
$\Theta(n)$	Linear complexity
$\Theta(n \log n)$	Linearithmic complexity
$\Theta(n^b)$	Polynomial complexity
$\Theta(b^n)$, where $b \geq 1$	Exponential complexity
$\Theta(n!)$	Factorial complexity

Constant
logarithmic
Linear
linearithmic
polynomial
exponential
Factorial

TABLE 2 The Computer Time Used by Algorithms.

<i>Problem Size</i>	<i>Bit Operations Used</i>					
<i>n</i>	$\log n$	<i>n</i>	$n \log n$	n^2	2^n	$n!$
10	3×10^{-11} s	10^{-10} s	3×10^{-10} s	10^{-9} s	10^{-8} s	3×10^{-7} s
10^2	7×10^{-11} s	10^{-9} s	7×10^{-9} s	10^{-7} s	4×10^{11} yr	*
10^3	1.0×10^{-10} s	10^{-8} s	1×10^{-7} s	10^{-5} s	*	*
10^4	1.3×10^{-10} s	10^{-7} s	1×10^{-6} s	10^{-3} s	*	*
10^5	1.7×10^{-10} s	10^{-6} s	2×10^{-5} s	0.1 s	*	*
10^6	2×10^{-10} s	10^{-5} s	2×10^{-4} s	0.17 min	*	*

Büyük- O (Big- O) Gösterimi

Tanım: f ve g , tamsayı kümesinden veya reel sayı kümesinden reel sayılara tanımlanmış olsun. $\mathbb{Z}^+ \rightarrow \mathbb{R}$

Eğer, $x > k$ olduğunda $|f(x)| \leq C|g(x)|$ oluyorsa ve bu eşitsizliği sağlayan C ve k gibi sabit sayılar varsa $f(x) = O(g(x))$ olmaktadır.

Örnek

Show that $f(x) = x^2 + 2x + 1$ is $O(x^2)$

Since when $x > 1$, $x < x^2$ and $1 < x^2$

$$0 \leq x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2 = 4x^2$$

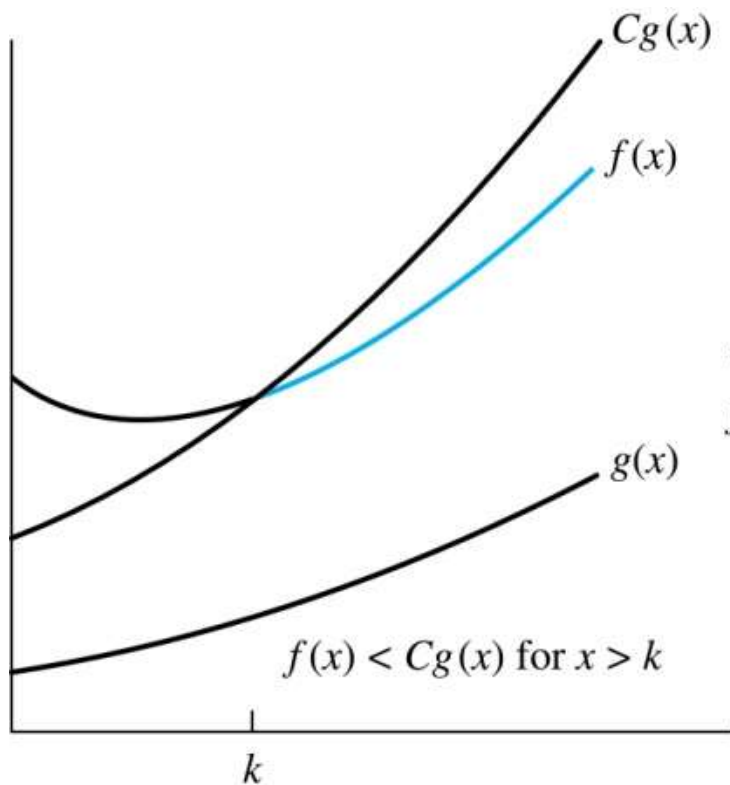
Can take $C = 4$ and $k = 1$ as witnesses to show that

$f(x)$ is $O(x^2)$

Alternatively, when $x > 2$, we have $2x \leq x^2$ and $1 < x^2$.

Hence, $0 \leq x^2 + 2x + 1 \leq x^2 + x^2 + x^2 = 3x^2$
when $x > 2$.

- Can take $C = 3$ and $k = 2$ as witnesses instead.



$f(x)$ is $O(g(x))$

The part of the graph of $f(x)$ that satisfies $f(x) < Cg(x)$ is shown in color.

ALGORITHM *SequentialSearch*($A[0..n - 1]$, K)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n - 1]$ and a search key K

//Output: The index of the first element of A that matches K

// or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

❑ Worst case $O(n)$

❑ Best case $O(1)$

❑ Average case $\frac{1+2+3+\dots+n}{n} = \frac{n(n+1)}{2n} = \frac{(n+1)}{2}$

Zaman karmaşıklığı	Açıklama	Örnek
$O(1)$	<u>Sabit</u> : Veri giriş boyutundan bağımsız gerçekleşen işlemler.	Bağlı listeye ilk eleman olarak ekleme yapma
$O(\log N)$	<u>Logaritmik</u> : Problemi küçük veri parçalarına bölen algoritmalarda görülür.	Binary search tree veri yapısı üzerinde arama
$O(N)$	<u>Lineer – doğrusal</u> : Veri giriş boyutuna bağlı doğrusal artan.	Sıralı olmayan bir dizide bir eleman arama
$O(N \log N)$	<u>Doğrusal çarpanlı logaritmik</u> : Problemi küçük veri parçalarına bölen ve daha sonra bu parçalar üzerinde işlem yapan.	N elemanı böl-parçala-yönet yöntemiyle sıralama. Quick Sort.
$O(N^2)$	Karesel	Bir grafikte iki düğüm arasındaki en kısa yolu bulma veya Buble Sort.
$O(N^3)$	Kübik	Ardarda gerçekleştirilen lineer denklemler
$O(2^N)$	İki tabanında üssel	Hanoi'nin Kuleleri problemi

Büyük- Ω (Big-*Omega*) Gösterimi

Tanım: f ve g , tamsayı kümesinden veya reel sayı kümesinden reel sayılara tanımlanmış olsun. $\mathbb{Z}^+ \rightarrow \mathbb{R}$

Eğer, $x > k$ olduğunda $|f(x)| \geq C|g(x)|$ oluyorsa ve bu eşitsizliği sağlayan C ve k gibi sabit sayılar varsa $f(x) = \Omega(g(x))$ olmaktadır.

Big-O ile Big- Ω arasında sıkı bir ilişki vardır.

Ancak ve ancak $g(x) = O(f(x))$ olduğunda $f(x) = \Omega(g(x))$ olacaktır.

Example: Show that $f(x) = 8x^3 + 5x^2 + 7$ is $\Omega(g(x))$ where $g(x) = x^3$.

Solution: $f(x) = 8x^3 + 5x^2 + 7 \geq 8x^3$ for all positive real numbers x .

- Is it also the case that $g(x) = x^3$ is $O(8x^3 + 5x^2 + 7)$?

Büyük- θ (Big-*Theta*) Gösterimi

Tanım: f ve g , tamsayı kümesinden veya reel sayı kümesinden reel sayılara tanımlanmış olsun. $\mathbb{Z}^+ \rightarrow \mathbb{R}$

Eğer, $f(x)$, $O(g(x))$ ve $f(x)$, $\Omega(g(x))$ ise $f(x)$, $\theta(g(x))$ deriz.

Eğer $x > k$ olduğunda $C_1|g(x)| \leq |f(x)| \leq C_2|g(x)|$ oluyorsa

ve bu eşitsizliği sağlayan pozitif C_1 ve C_2 reel sayıları ve bir

pozitif k reel sayısı bulunabiliyorsa bu durumda $f(x)$ 'in $\theta(g(x))$

olduğunu gösterebiliriz.

* Ex:

$$f(x) = 4x^3 + 3x^2, \quad g(x) = x^4$$

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = \frac{4x^3 + 3x^2}{x^4} = \lim_{x \rightarrow \infty} \frac{4x^3}{x^4} + \lim_{x \rightarrow \infty} \frac{3x^2}{x^4}$$

$$\underbrace{\lim_{x \rightarrow \infty} \frac{4}{x}}_{\emptyset} + \underbrace{\lim_{x \rightarrow \infty} \frac{3}{x^2}}_{\emptyset} = \emptyset$$

$L = \emptyset$ aktüjü isin,

$$4x^3 + 3x^2 \in O(x^4)$$

veya

$$4x^3 + 3x^2 \leq C \cdot (x^4) \text{ sâplâşın}$$

$C = 7$ gibî bir sayı verdî. $\therefore O(x^4)$ olur.

Ex | $n > 6$ ve n bir tam sayı iken $3n < n!$ i gösteriniz.

Tamamı olabilir veya Big O kullanılır.

1) $n = \{7, 8, 9, \dots\}$ kasesinden;

$n=7$ için

$$3 \cdot 7 < 7! \quad \text{doğru}$$

2) $n=k$ doğru kabul edilir.

$$3k < k! \quad \text{doğru kabul edilir.}$$

3) $n=k+1$?

$$3(k+1) < (k+1)! \quad \text{doğruluğu gösterilir,}$$

$$3k+3 < (k+1) \cdot k!$$

$$3k+3 < k \cdot k! + k!$$

$k! > 3k$ $k > 6$ iken
doğru kabul edildi

Burada $k \cdot k! > 3$ ispat edilmeli $k > 6$ iken bu
doğrudur.

$$\therefore 3n < n! \quad \text{doğrudur.}$$

Arama Algoritmaları

Searching

Algorithm	Data Structure	Time Complexity		Space Complexity
		Average	Worst	Worst
Depth First Search (DFS)	Graph of $ V $ vertices and $ E $ edges	-	$O(E + V)$	$O(V)$
Breadth First Search (BFS)	Graph of $ V $ vertices and $ E $ edges	-	$O(E + V)$	$O(V)$
Binary search	Sorted array of n elements	$O(\log(n))$	$O(\log(n))$	$O(1)$
Linear (Brute Force)	Array	$O(n)$	$O(n)$	$O(1)$
Shortest path by Dijkstra, using a Min-heap as priority queue	Graph with $ V $ vertices and $ E $ edges	$O((V + E) \log V)$	$O((V + E) \log V)$	$O(V)$
Shortest path by Dijkstra, using an unsorted array as priority queue	Graph with $ V $ vertices and $ E $ edges	$O(V ^2)$	$O(V ^2)$	$O(V)$
Shortest path by Bellman-Ford	Graph with $ V $ vertices and $ E $ edges	$O(V E)$	$O(V E)$	$O(V)$

Sıralama Algoritmaları

Sorting

Algorithm	Data Structure	Time Complexity			Worst Case Auxiliary Space Complexity
		Best	Average	Worst	Worst
Quicksort	Array	$O(n \log(n))$	$O(n \log(n))$	$O(n^2)$	$O(n)$
Mergesort	Array	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Heapsort	Array	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(1)$
Bubble Sort	Array	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	Array	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Select Sort	Array	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Bucket Sort	Array	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(nk)$
Radix Sort	Array	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$

Heap ve Graf Algoritmaları

Heaps

Heaps	Time Complexity						
	Heapify	Find Max	Extract Max	Increase Key	Insert	Delete	Merge
Linked List (sorted)		$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(m+n)$
Linked List (unsorted)		$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Binary Heap	$O(n)$	$O(1)$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(m+n)$
Binomial Heap		$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Fibonacci Heap		$O(1)$	$O(\log(n))^*$	$O(1)^*$	$O(1)$	$O(\log(n))^*$	$O(1)$

Graphs

[illegible]

Birçok algoritma birden fazla alt programdan oluşabilir.

$$\left. \begin{array}{l} f1 \rightarrow O(n^c) \\ f2 \rightarrow O(n^d) \\ 1 < c < d \text{ ise} \end{array} \right\} \max(O(n^c), O(n^d)) \rightarrow O(n^d)$$

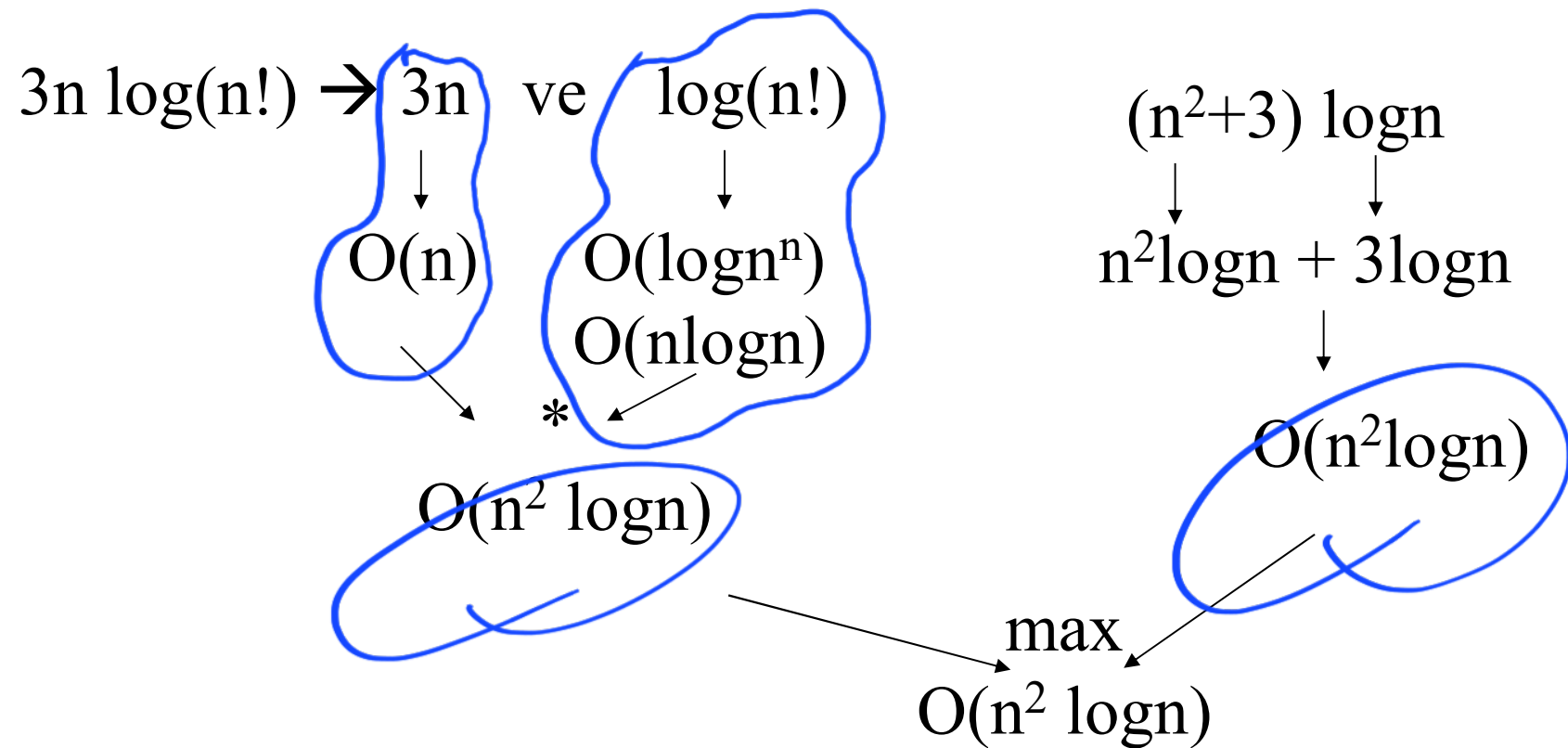
$$\left. \begin{array}{l} f1 \rightarrow O(\log n) \\ f2 \rightarrow O(n) \end{array} \right\} \max(O(\log n), O(n)) \rightarrow O(n)$$

$$\left. \begin{array}{l} f1 \rightarrow O(2^n) \\ f2 \rightarrow O(n) \end{array} \right\} \max(O(2^n), O(n)) \rightarrow O(2^n)$$

$$\left. \begin{array}{l} f1 \rightarrow O(n^2) \\ f2 \rightarrow O(n^2) \end{array} \right\} \rightarrow O(n^2)$$

$$f(n) = 3n \log(n!) + (n^2+3) \log n$$

n , pozitif bir tamsayı olmak üzere Big-O ?



$$F(x) = (x+1) \log(x^2+1) + 3x^2 \quad \text{Big-O ?}$$

$$O(x+1) \rightarrow O(x)$$

$$O(\log(x^2+1)) \rightarrow O(\log x^2) \rightarrow O(2\log x) \rightarrow O(\log x)$$

$$O(x \log x)$$

$$3x^2 \rightarrow O(3x^2) \rightarrow O(x^2)$$

$$\max(O(x \log x), O(x^2)) \rightarrow O(x^2)$$

İteratif ve Özyinelemeli Algoritmaların Analizi



İteratif (nonrecursive) algoritmaların analizi

Genel Adımlar:

- n girdi boyutu (*input size*) belirlenir
- Algoritmanın temel operasyonu saptanır (*basic operation*)
- Durum analizleri (worst, average ve best cases for input of size) n değerine göre belirlenir
- Temel operasyonun kaç kez işletileceğini hesaplamak için toplama işlemi yapılır ve n değerine bağlı bir çalışma zamanı fonksiyonu $T(n)$ elde edilir
- Toplama sonucu elde edilen n 'e bağlı fonksiyon $T(n)$, asimptotik notasyonlara göre ifade edilir

Insertion Sort (Sokma Sıralaması)

A *pseudocode* for insertion sort (INSERTION SORT)

INSERTION-SORT(A)

```
1  for  $j \leftarrow 2$  to  $\text{length}[A]$ 
2      do  $\text{key} \leftarrow A[j]$ 
3      Insert  $A[j]$  into the sorted sequence  $A[1, \dots, j-1]$ .
4       $i \leftarrow j - 1$ 
5      while  $i > 0$  and  $A[i] > \text{key}$ 
6          do  $A[i+1] \leftarrow A[i]$ 
7               $i \leftarrow i - 1$ 
8   $A[i + 1] \leftarrow \text{key}$ 
```

11	7	15	3	16	13
0	1	2	3	4	5

(key indisi = 1)

11	7	15	3	16	13
0	1	2	3	4	5

7	11	15	3	16	13
0	1	2	3	4	5

7	11	15	3	16	13
0	1	2	3	4	5

(key indisi = 2)

7	11	15	3	16	13
0	1	2	3	4	5

7	11	15	3	16	13
0	1	2	3	4	5

(key indisi = 3)

7	11	15	3	16	13
0	1	2	3	4	5

3	7	11	15	16	13
0	1	2	3	4	5

3	7	11	15	16	13
0	1	2	3	4	5

(key indisi = 4)

3	7	11	15	16	13
0	1	2	3	4	5

3	7	11	15	16	13
0	1	2	3	4	5

(key indisi = 5)

3	7	11	15	16	13
0	1	2	3	4	5

3	7	11	13	15	16
0	1	2	3	4	5

3	7	11	13	15	16
0	1	2	3	4	5

Toplam Çalışma süresi

$$T(n) = c_1 + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\ + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1).$$

Best-case : $O(n)$

Dizi başlangıçta sıralı ise. Yer değiştirme yapılmaz

Average-case : $O(n^2)$

Worst case : $O(n^2)$

Başlangıçta dizi büyükten küçüğe sıralı ise her eleman için **en başa** kadar karşılaştırma yapılacaktır.

Selection Sort (Seçmeli Sıralama)

```
procedure selection sort
  list  : array of items
  n     : size of list

  for i = 1 to n - 1
    /* set current element as minimum*/
    min = i

    /* check the element to be minimum */

    for j = i+1 to n
      if list[j] < list[min] then
        min = j;
      end if
    end for

    /* swap the minimum element with the current element*/
    if indexMin != i then
      swap list[min] and list[i]
    end if
  end for

end procedure
```

17	33	14	27	18
0	1	2	3	4

Swap Yapılır. (**i = 0**)

17	33	14	27	18
i = 0	1	2	3	4

14	33	17	27	18
i = 0	1	2	3	4

Swap Yapılır. (**i = 1**)

14	33	17	27	18
----	----	----	----	----

14	17	33	27	18
----	----	----	----	----

Swap Yapılır. (**i = 2**)

14	17	33	27	18
0	1	2	3	4

14	17	18	27	33
0	1	2	3	4

Swap Yapılır. (**i = 3**)

14	17	18	27	33
0	1	2	3	4

```

void selection(int A[], int N)
{ int i, j, temp;
  for (j = 0; j < N-1; j++)  $\xrightarrow{\text{iterations}}$  (N-1)
  { int min_idx = j;
    for (i = j+1; i < N; i++)  $\xrightarrow{\text{iterations}}$  (N-1-j)
      if (A[i] < A[min_idx]) (depends on i)
        min_idx = i;
    temp = A[min_idx];
    A[min_idx] = A[j];
    A[j] = temp;
  }
}

```

j	Iterations of inner loop = N-1-j
0	N-1
1	N-2
2	N-3
...	...
N-3	2
N-2	1

Total instructions (all iterations of inner loop for all values of j)
 $T(N) = (N-1) + (N-2) + \dots + 2 + 1 =$
 $= [N * (N-1)] / 2 \rightarrow N^2 \text{ order of magnitude}$
Note that the came from the summation NOT because 'there is an N in the inner loop' (NOT because $N * N$).

worst-case : $O(n^2)$
 average-case : $O(n^2)$
 best_case : $O(n^2)$

Öz yinelemeli (Recursive) Algoritmaların Analizi

- ❑ n girdi boyutu (*input size*) belirlenir
- ❑ Algoritmanın temel operasyonu saptanır (*basic operation*)
- ❑ Temel işlemin gerçekleştirilme sayısının aynı boyuttaki farklı girişlere göre değişiklik gösterip göstermediği kontrol edilir (Eğer değişiklik gösterirse, durum analizi ayrı ayrı the worst, average ve best cases yapılmalıdır)
- ❑ Temel işlemin kaç kez yürütüldüğünü ifade eden uygun bir başlangıç koşulu ile bir özyineleme bağıntısı (recurrence relation) kurulur
- ❑ Recurrence relation uygun bir yöntemle çözülür. Çözüm sonucu elde edilen n 'e bağlı fonksiyon $T(n)$, asimptotik notasyonlara göre ifade edilir

Öz yinelemeli (Recursive) bağıntılarının çözümü

- n bağımsız değişkene göre ifade edilen ve n değerinin önceki değerlerini içinde bulunduran fonksiyonlardır

$$T(n) = T(n-1) + 1$$

- Çözüm yöntemleri
 - Forward Substitution 
 - Backward Substitution
 - Recursive Tree Method
 - Master Theorem
 - Karakteristik Denklem Yöntemi (2. veya yüksek dereceden bağıntılar için)

Forward Substitution

- Recurrence: $T(n) = T(n-1) + 1$, with initial condition $t(1) = 2$
- Look for a pattern:
 - $T(1) = 2$, Initial condition
 - $T(2) = T(1) + 1 = 2 + 1 = 3$
 - $T(3) = T(2) + 1 = 3 + 1 = 4$
 - $T(4) = T(3) + 1 = 4 + 1 = 5$
 - $T(5) = T(4) + 1 = 5 + 1 = 6$
- Guess:
 - $T(n) = n + 1$
- Informal Check:
 - $T(n) = T(n-1) + 1 = [(n-1) + 1] + 1 = n + 1$
 - $T(1) = 1 + 1 = 2$

Age Group	Percentage
18-24	10%
25-34	20%
35-44	25%
45-54	20%
55-64	15%
65-74	10%
75-84	5%
85+	5%



Gauss

$f(n)=n + f(n-1)$ $F(1)=1$ ilk değ er

$f(5) = 5 + f(4)$ 10
15

$f(4) = 4 + f(3)$ 6
10

$f(3) = 3 + f(2)$ 3
6

$f(2) = 2 + f(1)$ 1
3

Örnek : n sayısının faktöriyelini hesaplayalım

```
factorial(n):  
if n is 0  
return 1  
return n * factorial(n-1)
```

- factorial(0) sadece 1 karşılaştırma (1 unit of time)
- factorial(n) (1 karşılaştırma, 1 multiplication, 1 subtraction) (n-1) kez

$$T(n) = T(n - 1) + 3$$
$$T(0) = 1$$

n yerine n-1 koyuyoruz

$$\begin{aligned}T(n) &= T(n-1) + 3 \\&= T(n-2) + 6 \\&= T(n-3) + 9 \\&= T(n-4) + 12 \\&= \dots \\&= T(n-k) + 3k\end{aligned}$$

as we know $T(0) = 1$

we need to find the value of k for which $n - k = 0$, $k = n$

$$\begin{aligned}T(n) &= T(0) + 3n, k = n \\&= 1 + 3n\end{aligned}$$

that gives us a time complexity of $O(n)$

↳ Bir dizinin içinde kendinden

bir parça bulunuyorsa
ona rekürans bağıntısı denir

Rekürans Bağıntıları (Özyineli Bağıntılar)

Bir dizinin içinde kendisinden bir parça bulunuyorsa o diziye rekürans bağıntısı (özyineli dizi) denir.

$$a_n = 5n + 1 \quad (\text{dizi}) \quad (\text{sequence}) \quad \rightarrow a_{20} = 101$$

$$\boxed{a_n = 2a_{n-1} + 3} \quad (a_{n-1} \text{ kendisinden parça}) \quad \text{rekürans bağıntısı}$$

$\downarrow$
 $a_0 = 3 \quad n \geq 1$

$$\left. \begin{array}{l} a_n = a_{n-1} - 2a_{n-2} + 5^n \\ a_n = 3a_{n-1} + 2a_{n-2} \end{array} \right\} \rightarrow a_1 = 3, a_0 = 5, n \geq 2$$

rekürans bağıntısı

Ör: $a_n = 2a_{n-1} - a_{n-2} + 2^{n-1}$, $a_0 = 1, a_1 = 2, n \geq 2$ olarak veriliyor. Buna göre, a_5 nedir?

$$n=2 \Rightarrow a_2 = 2a_1 - a_0 + 2 \Rightarrow a_2 = 4 - 1 + 2 \Rightarrow \boxed{a_2 = 5}$$

$$n=3 \Rightarrow a_3 = 2a_2 - a_1 + 4 \Rightarrow a_3 = 10 - 2 + 4 \Rightarrow \boxed{a_3 = 12}$$

$$n=4 \Rightarrow a_4 = 2a_3 - a_2 + 8 \Rightarrow a_4 = 24 - 5 + 8 \Rightarrow \boxed{a_4 = 27}$$

$$n=5 \Rightarrow a_5 = 2a_4 - a_3 + 16 \Rightarrow a_5 = 54 - 12 + 16 \Rightarrow a_5 = 58 //$$

Ayrık matematikteki hedefimiz

Rekürans Bağıntısı

Rekürans bağıntısını çözmek

$$a_n = 3a_{n-1} + a_{n-2}$$

$$a_n = 2^n - n + 1$$

İç bileşenlerden kurtaracak şekilde yazmalıyız

→ Bu rekürans
bağıntısı
depildir

Sözel bir problemi rekürans
bağıntısına çevirme

Rekürans Bağıntılarının Sınıflandırılması

a_n 'nin kendisinden sonraki kaç terime bağlı olması rekürans bağıntısının derecesini verir

1) Derece (Order)

$$a_n = 5a_{n-1} + n^2$$

→ 1 first order

$$a_n = 3a_{n-1} - 5a_{n-2}$$

→ 2 second order

$$a_n = 3a_{n-1} + a_{n-3} + 3^n$$

→ 3 third order

$$a_n = 2a_{n-4} + 1$$

→ 4. derece

2) Homojen – homojen değil (homogeneous – nonhomogeneous)

$$a_n = 3a_{n-1} \quad \checkmark \text{ Homojen}$$

$$a_n = a_{n-1} - 2a_{n-2} \quad \checkmark \text{ Homojen}$$

$$a_n = 3a_{n-1} + n \cdot a_{n-2} + 3^n \quad \times \text{ Homojen değil}$$

$$a_n = a_{n-1} + 2a_{n-3} - 4 \quad \times$$

$$a_n = a_{n-2} + n^3 + n^2 \quad \times$$

rekürans bağıntıları
homojen de olabilir
homojen olmaya daabilir

2. rekürans bağıntılarının homojenlik testiyle çarpılması

Lineerde a_n lerin üslerinin alınması ve a_n 'li terimlerin alınması lineerliği bozar

3) Lineer, lineer olmayan (Linear, nonlinear)

a_n 'li terimlerin üslerinin alınması veya birbirleri ile çarpılması lineerliği bozar.

$$a_n = 5a_{n-2} + n \cdot a_{n-3} + n^2 \quad \checkmark$$

$$a_n = (a_{n-1})^2 + 3n \quad \times$$

$$(a_n)^3 = a_{n-1} + a_{n-2} \quad \times$$

$$a_n = (a_{n-1}) \cdot (a_{n-2}) - 3 \quad \times$$

4) Sabit Katsayılı olan ve olmayan (constant coefficient)

a_n 'li ifadeler n 'li birşey ile çarpım durumunda olursa sabit katsayılı değildir.

$$a_n = 3a_{n-1} + 5a_{n-2} + n - 7 \quad \checkmark$$

$$a_n = n \cdot a_{n-1} + 3a_{n-2} + 5^n \quad \times$$

$$a_n = 7a_{n-1} + 10a_{n-2} \quad \checkmark$$

$$a_n = 3^n \cdot a_{n-2} + 1 \quad \times$$

Sabit Katsayılı Homojen Rekürans Bağlıntılarının Çözülmesi (Karakteristik Kök Tekniği)

Karakteristik Kök Tekniği

- İkinci derece ve daha büyük dereceli rekürans bağlıntılarının çözümünde kullanılır.
- Sabit ve homojen rekürans bağlıntılarında kullanılır.
- Homojen olmayan rekürans bağlıntısında ise homojen hale getirmek için kullanılır.

$$r^2 \quad r \quad 1$$

$$a_n = 3a_{n-1} + 4a_{n-2}$$

Sabit Katsayılı Homojen 2. derece
bir rekürans bağıntısı

Homojen olduğundan
 $a_n = a_n^{(h)}$

homojen olmadığında
 $a_n = a_n^{(h)} + a_n^{(ö)}$

$$a_{n-2} = 1$$

$$a_{n-1} = r$$

$$a_n = r^2$$

$$r^2 = 3r + 4$$

$$r^2 - 3r - 4 = 0$$

karakteristik denklem

2 tane farklı kökü varsa

Eşit 2 tane kökü varsa

eger 2 tane
kökümüz
varsa bunlar ya
birbirinden farklı

olabilir ya da aynı olabilir

Karakteristik Denklemin Köklerine Göre Homojen Çözümün Yazımı

2. derece

$$1) r_1 \neq r_2 \text{ ise } \Rightarrow$$

$$a_n^{(h)} = c_1 \cdot (r_1)^n + c_2 \cdot (r_2)^n$$

→ Kökler birbirinden farklı olursa

$$2) r_1 = r_2 \text{ ise } \Rightarrow$$

$$a_n^{(h)} = c_1 \cdot (r_1)^n + c_2 \cdot (r_2)^n \cdot n$$

→ Kökler aynı olursa

$$r=4 \quad r=-1 \\ (r-4)(r+1)=0$$

$$r^2 - 3r - 4 = 0$$

$$a_n = c_1 \cdot 4^n + c_2 \cdot (-1)^n$$

3. derece

$$1) r_1 \neq r_2 \neq r_3 \Rightarrow a_n^{(h)} = c_1 \cdot (r_1)^n + c_2 \cdot (r_2)^n + c_3 \cdot (r_3)^n$$

$$2) r_1 = r_2 = r_3 \Rightarrow a_n^{(h)} = c_1 \cdot (r_1)^n + c_2 \cdot (r_2)^n \cdot n + c_3 \cdot (r_3)^n \cdot n^2$$

$$3) r_1 = r_2 \neq r_3 \Rightarrow a_n^{(h)} = c_1 \cdot (r_1)^n + c_2 \cdot (r_2)^n \cdot n + c_3 \cdot (r_3)^n$$

Soru: $a_n = 3a_{n-1} + 10a_{n-2}$, $a_0 = 5$ ve $a_1 = 8$ olan bağımlı değerleri
verilmiş rekürans bağıntısını çözünüz.

1. adım $a_n = a_n^{(h)}$

2. adım $r^2 = 3r + 10$ (karakteristik denklemler)

2. derece $r^2 - 3r - 10 = 0$
 $(r-5)(r+2) = 0$
5 -2

$$a_n^{(h)} = c_1 (5)^n + c_2 (-2)^n$$

3. adım

$$a_n = c_1 5^n + c_2 (-2)^n$$

$$a_0 = 5 \Rightarrow \boxed{c_1 + c_2 = 5} \quad / 2$$

$$a_1 = 8 \Rightarrow \boxed{5c_1 - 2c_2 = 8}$$

$$7c_1 = 18$$

$$\boxed{c_2 = \frac{17}{7}}$$

$$a_n = \frac{18}{7} \cdot 5^n + \frac{17}{7} \cdot (-2)^n$$

Soru: $a_n = -4a_{n-1} - 4a_{n-2}$, $a_0 = 3$, $a_1 = 5$ başlangıç koşulları ile verilen rekürans bağıntısının çözümünü bulunuz.

1. adım $a_n = a_n^{(h)}$

2. adım $a_{n-2} = 1$

2. derece $a_{n-1} = r \Rightarrow r^2 = -4r - 4 \Rightarrow r^2 + 4r + 4 = 0$
 $a_n = r^2$
 $(r+2)(r+2) = 0$
 $r_1 = -2$ $r_2 = -2$

$$a_n^{(h)} = c_1 \cdot (-2)^n + c_2 (-2)^n \cdot n$$

3. adım $a_n = c_1 (-2)^n + c_2 (-2)^n$

$a_0 = 3 \Rightarrow \boxed{c_1 = 3}$

$$a_n = 3 \cdot (-2)^n - \frac{11}{2} (-2)^n \cdot n$$

$a_1 = 5 \Rightarrow -2c_1 - 2c_2 = 5 \Rightarrow -2c_2 = 11 \Rightarrow \boxed{c_2 = -\frac{11}{2}}$

Sabit Katsayılı Homojen Olmayan Rekürans Bağlıntılarının Çözülmesi

$$a_n = 3a_{n-1} + 4a_{n-2} + f(n)$$

genel

özel

$$a_n^{(g)} = a_n^{(h)} + a_n^{(ö)}$$

polinom olursa

1

3

$n+2$

n^2

Üstel fonksiyon olursa

$3 \cdot 2^n$

5^n

7^{-n}

Özel çözüm bulunurken $a_n^{(ö)}$ tahmin edilir ve çözüm bu şekilde elde edilir.

Polinom Durumu

Ör: $a_n = 2a_{n-1} + 3a_{n-2} + 5$, $a_0 = 4$, $a_1 = 10$ başlangıç koşulları ile verilen rekürans bağıntısını çözümlüyoruz.

1. adım $a_n^{(g)} = a_n^{(h)} + a_n^{(ö)}$

2. adım $a_n^{(h)}$ bulunur.

$$a_n = 2a_{n-1} + 3a_{n-2} \Rightarrow r^2 = 2r + 3$$

$\uparrow \quad \uparrow \quad \uparrow$
 $r^2 \quad r \quad 1$

$$r^2 - 2r - 3 = 0$$

$$(r-3)(r+1) = 0$$

$$r = 3 \quad r = -1$$

$$a_n^{(h)} = C_1 \cdot 3^n + C_2 \cdot (-1)^n$$

$$a_n = 2a_{n-1} + 3a_{n-2} + 5$$

not: polinomun derecesi ne ise a_n nin de dereceside aynı olur
f(n) sabit olduğundan tüm terimler sabite eşit olur

3. adım $a_n^{(0)}$ tahmin edilir.

$$a_n^{(0)} = A$$

$$a_{n-1} = A$$

$$a_{n-2} = A$$

ardından a_{n-1} ve a_{n-2}
elde edilip eşitlikte
yerine konular, ana a_n
tahmindeki katsayıları
bulmak için.

$$A = 2A + 3A + 5 \Rightarrow -4A = 5 \Rightarrow A = -\frac{5}{4}$$

$$a_n^{(0)} = -\frac{5}{4}$$

4. adım :

$$a_n = c_1 \cdot 3^n + c_2 \cdot (-1)^n - \frac{5}{4}$$

$$a_0 = 4$$

$$a_1 = 10$$

$$c_1 + c_2 - \frac{5}{4} = 4 \Rightarrow c_1 + c_2 = \frac{21}{4}$$

$$3c_1 - c_2 - \frac{5}{4} = 10 \Rightarrow 3c_1 - c_2 = \frac{45}{4}$$

$$\Rightarrow 4c_1 = \frac{45}{4} + \frac{33}{2}$$

$$c_1 = \frac{33}{8}$$

$$c_2 = \frac{9}{8}$$

$$a_n = \frac{33}{8} \cdot 3^n + \frac{9}{8} (-1)^n - \frac{5}{4}$$

Ör: $a_n = a_{n-1} + 2a_{n-2} + 3n + 4$, $a_0 = 5$, $a_1 = 7$ başlangıç koşulları ile verilen rekürans bağıntısını çözelim.

1. adım $a_n^{(g)} = a_n^{(h)} + a_n^{(ö)}$

2. adım $a_n^{(h)} \rightarrow a_n = a_{n-1} + 2a_{n-2} \Rightarrow r^2 = r + 2$

$$r^2 - r - 2 = 0$$

$$(r-2)(r+1) = 0$$

$$2 \quad -1$$

$$a_n^{(h)} = c_1 \cdot 2^n + c_2 \cdot (-1)^n$$

1. derece polinom

$a_n = a_{n-1} + 2a_{n-2} + 3n + 4$

3. adım $a_n^{(ö)} = An + B$

$a_{n-1} = A(n-1) + B$

$a_{n-2} = A(n-2) + B$

$$\cancel{An+B} = \cancel{A(n-1)+B} + 2\cancel{A(n-2)+B} + 3n+4$$

$$\underline{\underline{-2An + 5A - 2B = 3n + 4}}$$

$$\boxed{A = -\frac{3}{2}}$$

$$5A - 2B = 4 \Rightarrow -\frac{15}{2} - 2B = 4$$

$$-\frac{23}{2} = 2B \Rightarrow \boxed{B = -\frac{23}{4}}$$

$$\boxed{a_n^{(ö)} = -\frac{3}{2}n - \frac{23}{4}}$$

Burdaki
n yerine
n-1 koyor
ve garpöriz
sonuç olarak
do bu çıkar

4.adım

$$a_n = c_1 \cdot 2^n + c_2 \cdot (-1)^n - \frac{3}{2}n - \frac{23}{4}$$

$$a_0 = 5$$

$$a_1 = 7$$

c_1 ve c_2 bulunup en son a_n yazılır.

Üstel Fonksiyon Durumu

Ör: $a_n = -4a_{n-1} - 3a_{n-2} + 2 \cdot 5^n$, $a_0 = 6$, $a_1 = 10$ başlangıç koşulları ile verilen rekürans bağıntısını çözümlü.

1. adım: $a_n^{(g)} = a_n^{(h)} + a_n^{(ö)}$

2. adım $a_n^{(h)} \Rightarrow a_n = -4a_{n-1} - 3a_{n-2} \Rightarrow r^2 = -4r - 3$

$$r^2 + 4r + 3 = 0$$

$$(r+3)(r+1) = 0$$

$$r = -3 \quad r = -1$$

$$a_n^{(h)} = c_1(-3)^n + c_2 \cdot (-1)^n$$

$a_n^{(h)}$ içinde var mı?

YOK

Eğer varsa

1 tane varsa

$a_n^{(ö)} \rightarrow 1$ tane

n carpanı gelir.

2 tane varsa

$a_n^{(ö)} \rightarrow 2$ tane

n^2 carpanı gelir.

3. adım $a_n^{(ö)} = A \cdot 5^n$

$$a_{n-1} = A \cdot 5^{n-1} = \frac{A}{5} \cdot 5^n$$

$$a_{n-2} = A \cdot 5^{n-2} = \frac{A}{25} \cdot 5^n$$

$$A \cdot 5^n = -\frac{4A}{5} 5^n - \frac{3A}{25} 5^n + 2 \cdot 5^n$$

$$A = -\frac{23A}{25} + 2 \Rightarrow \frac{48A}{25} = 2$$

$$A = \frac{25}{24}$$

$$a_n^{(ö)} = \frac{25}{24} \cdot 5^n$$

4. adım:

$$a_n = c_1 \cdot (-3)^n + c_2 \cdot (-1)^n + \frac{25}{24} \cdot 5^n$$

$$a_0 = 6$$

$$a_1 = 10$$

$$c_1 + c_2 + \frac{25}{24} = 6$$

$$\Rightarrow c_1 = \dots$$

$$-3c_1 - c_2 + \frac{125}{24} = 10$$

$$c_2 = \dots$$

$$a_n = \dots$$

Ör: $a_n = 2a_{n-1} + 3a_{n-2} + 3^n$, $a_0 = 7$, $a_1 = 5$ çözümlü.

1. adım $a_n^{(3)} = a_n^{(h)} + a_n^{(ö)}$ 1 tane var.

2. adım $a_n^{(h)} \Rightarrow a_n = 2a_{n-1} + 3a_{n-2} \Rightarrow r^2 - 2r - 3 = 0$

$$(r-3)(r+1) = 0$$

$$r=3 \quad r=-1$$

$$a_n^{(h)} = c_1 3^n + c_2 (-1)^n$$

3. adım $a_n^{(ö)} = A \cdot 3^n \cdot n$

$$a_{n-1} = A \cdot 3^{n-1} \cdot (n-1) = \frac{A}{3} \cdot 3^n \cdot n - \frac{A}{3} \cdot 3^n$$

$$a_{n-2} = A \cdot 3^{n-2} \cdot (n-2) = \frac{A}{9} \cdot 3^n \cdot n - \frac{2A}{9} \cdot 3^n$$

$$\cancel{A \cdot 3^n \cdot n} = \cancel{\frac{2A}{3} \cdot 3^n \cdot n} - \cancel{\frac{2A}{3} \cdot 3^n} + \cancel{\frac{A}{3} \cdot 3^n \cdot n} - \cancel{\frac{2A}{3} \cdot 3^n} + 3^n$$

$$A \cdot 3^n \cdot n$$

$$\frac{4A}{3} \cdot 3^n = 3^n$$

$$\frac{4A}{3} = 1 \quad A = \frac{3}{4}$$

$$a_n^{(ö)} = \frac{3}{4} \cdot 3^n \cdot n$$

Sayı Dizilerini Rekürans Bağıntısına Dönüştürme

Bizlere bazen bir sayı dizisi bazen de sözel bir ifade verilerek ondan bir sayı dizi formunda olayı yazmamız beklenir.

4, 7, 13, 25, 49, 97, Sayı dizisi

Rekürans
bağıntısı :

$$a_0 = 4, \quad a_n = 2a_{n-1} - 1, \quad n \geq 1$$

$$r-t=0 \\ r=t$$

$$a_n = A \\ a_{n-1} = A$$

$$A = 2A - 1$$

Ör: 1, 3, 7, 17, 41, 99, 239, ... sayı dizisinin rekürans bağıntısını
olarak yazınız.

$$\begin{aligned} a_0 &= 1 \\ a_1 &= 3 \end{aligned}$$

$$a_n = 2a_{n-1} + a_{n-2}, \quad n \geq 2$$

Homojen olmayan rekürans bağıntısı için bir örnek yazalım

Ör: $1, 3, 6, 13, 27, 56, 115, \dots$ sayı dizisini rekürans bağıntısı haline getiriniz.

$1+3+2$
 $3+6+4$
 $6+13+8$

$a_0 = 1$
 $a_1 = 3$

$a_n = a_{n-1} + a_{n-2} + 2^{n-1}, n \geq 2$

Özel fonksiyon olarak

homojen olarak

$a_2 = 2a_1 + a_0 \quad a_2 = 7$
 $a_3 = 2a_2 + a_1 \quad a_3 = 17$

Soru: Sadece 0 ve 1'lerden oluşan n birim uzunluğunda sayı dizileri oluşturuluyor. Buna göre, bu sayı dizilerinden n uzunluğunda olup "00" içerenlerin sayısını veren rekürrens bağıntısını başlangıç koşullarını da belirleyerek yazınız.

$n = 0$ birim	→ dizi yok	$a_0 = 0$
$n = 1$ "	→ 0, 1	$a_1 = 0$
$n = 2$ "	→ 00, 01, 10, 11	$a_2 = 1$
$n = 3$ "	→ <u>000</u> , <u>001</u> , 010, <u>100</u> 011, 101, 110, 111	$a_3 = 3$
$n = 4$ "	→ 0000- 0011- 0111 0001- 0101 1011 0010- 1001- 1101 0100- 0110 1110 1000- 1010 1111 1100-	$a_4 = 8$

0, 0, 1, 3, 8, - - - - -

$$a_n = 2 \cdot a_{n-1} + a_{n-2} + 1 \quad n \geq 2$$

$$a_1 = 0$$

$$a_0 = 0$$

Problemlerin Karmaşıklığı

Çözülebilir Problemler (Tractable)

- ◆ Polinomsal en kötü durum karmaşıklığına sahip bir algoritma kullanarak çözülebilen bir problem **çözülebilir** (tractable) olarak adlandırılmaktadır.
- ◆ Çünkü algoritmanın, nispeten kısa bir zaman içinde makul büyüklükte veriye sahip bir probleme çözüm getireceği beklenmektedir.
- ◆ Bununla birlikte, büyük- O tahminindeki polinomlar yüksek dereceye sahipse (100. derece gibi) veya katsayılar aşırı büyükse, algoritmanın problemi çözmesi oldukça uzun zaman alabilir.
- ◆ Sonuç olarak, polinomsal en kötü durum zaman karmaşıklığına sahip bir algoritma kullanarak çözülebilen bir problemin nispeten küçük veri değerlerinde bile makul zamanda çözülebilmesi garanti edilemez.
- ◆ Pratikte bu tür tahminlerdeki polinomların derece ve katsayıları genellikle küçüktür.

Çözülemez Problemler (Intractable)

- ◆ En kötü durum polinomsal zaman karmaşıklığına sahip bir algoritma kullanılarak çözilemeyen problemler **çözülemez** (intractable) olarak adlandırılmaktadır.
- ◆ En kötü duruma sahip bir problemin çözümü, çok küçük girdi değerleriyle bile her zaman olmasa da genellikle aşırı miktarda zamana ihtiyaç duyabilmektedir.
- ◆ Bununla birlikte, pratikte bazı en kötü durum zaman karmaşıklığına sahip algoritmaların bir problemi, birçok durumda kendisine özgü en kötü durumundan çok daha hızlı çözebildiği durumlar vardır.
- ◆ Muhtemelen çok az sayıdaki durumların makul zamanda çözilememesine izin vermeye istekli olduğumuzda, ortalama durum zaman karmaşıklığı, bir algoritmanın bir problemi ne kadar uzun zamanda çözeceğinin en iyi ölçüsüdür.
- ◆ Endüstride önemli olan birçok problem çözülemez olarak düşünülmektedir ancak uygulamada, aslında günlük yaşamın getirdiği tüm girdi durumları için çözülebilmektedir.
- ◆ Pratik uygulamada karşılaşılan çözülemez problemlerin üstesinden gelmenin bir diğer yolu da problemin kesin çözümünü bulmak yerine yaklaşık çözümler aramaktır.
- ◆ Bu tür yaklaşık çözümler bulmakta hızlı algoritmaların varlığı işe yaramakta ve hatta kesin çözümden çok farklı olmama olasılığı bulunmaktadır.

Çözümü Bulunmayan Alg. (Unsolvable)

- ◆ Hiçbir algoritmanın onları çözemediği bazı problemler mevcuttur.
- ◆ Bu tür problemler **çözümü bulunamayan (unsolvable)** olarak adlandırılmaktadır
- ◆ Algoritma kullanarak **çözülebilir** problemlerin zıttı olarak ifade edilebilir.
- ◆ Çözümü bulunamayan problemlerin varlığına ilk kanıt, sonlanma (halting) probleminin çözülemez olduğunu gösteren büyük İngiliz matematikçi ve bilgisayar bilimcisi Alan Turing tarafından sağlanmıştır.

P ve NP Problem Sınıfları

- ◆ Çözülebilir problemlerin **P sınıfına (Class P)** ait olduğu söylenir.
 - ◆ Polinomsal zamanda kontrol edilebilen bir çözüm için problemlerin **NP sınıfına (Class NP)** ait olduğu söylenmektedir.
 - ◆ NP kısaltması, *belirli olmayan polinomsal zamanlı (nondeterministic polynomial time)* anlamına gelmektedir.
-
- ❖ Problemlerin herhangi birisi polinomial en kötü-durum zaman algoritmaları ile çözülebildiğinde, NP sınıfındaki tüm problemler de polinomsal en kötü-durum zaman algoritmaları ile çözülebilir. Bu özelliği taşıyan algoritmalara **NP-tam problemler (NP-complete problems)**
 - ❖ **NP-Hard**, polinomsal zamanda bir çözümü olduğunu ispatlayamadığımız karar problemlerinin karmaşıklık sınıfıdır.

Kaynaklar

- Levitin “Introduction to the Design & Analysis of Algorithms,” 3rd ed., Ch. 1 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved
- <https://www.javatpoint.com>
- <https://algorithms.tutorialhorizon.com>
- <https://www.tutorialspoint.com/>